

Temperature data option calculation

Pietro Rota

Università cattolica del sacro cuore

Table of contents

1 Weather Derivatives Temperature Options	1
2 Climate data download	4
3 Initial data visualization	7
3.1 Overlap of the dataset	11
3.2 Seasonal analysis	14
3.3 Long term trends	18
4 Seasonal decomposition	19
4.1 Model performance	23
4.2 Visualization of results	23
4.3 Residuals analysis and diagnostics	28
5 Ornstein-Uhlenbeck (OU) process	31
6 Modeling volatility	33
6.1 Polynomial Regression:	34
6.2 Fourier transforms	36
6.3 B-splines	38
6.4 Kernel smoothing	40
6.5 Loess modelling	41
6.6 GAM	43
6.7 Figuring out which is the best model	45
7 Montecarlo simulations	48
8 Risk neutral pricing	58
8.1 Market model	61
8.2 Greek Numerical Approximations	66
9 Sidequests	69
9.1 Sidequest: predicting temperatures in the future	69
9.2 Milan comparison	71

1 Weather Derivatives Temperature Options

Last updated: 2025-10-27

Full dependencies list, click to expand list

```
[1] "R version 4.4.3 (2025-02-28 ucrt)"
```

```
[1] "x86_64-w64-mingw32/x64"
```

Functions loaded from main

```
[1] "document is not rendering"
```

Packages required to run this file

```
[1] "document is not rendering"
```

Functions defined in this document

```
[1] "document is not rendering"
```

```
[1] "document is not rendering"
```

```
DEBUG: functions_loaded returned: character
DEBUG: functions_loaded returned: character
DEBUG: Found 12 custom functions: bprint, check_acc, compare_cdf, desc_df,
extract_r_code, find_outliers, quickbarplot, quickplot, remove_outliers, RSS,
show_df, smart_round
DEBUG: Sourcing main file...
DEBUG: Functions available in main file: ADJR2, approx_fraction, basic_vol,
before_dot, before_triple_dot, black_scholes, bond_price, bprint, BS_full,
check_acc, clean_and_convert, compare_cdf, delog, desc_df, dump_functions,
estvol, EVEN, extract_r_code, extract_r_code_backup, extract_r_code_complex,
extract_r_code_simple, find_outliers, fixnames, format_dataframe,
format_number, forward_price, functions_loaded, get_portfolio, gg_qq_plot,
greeks, ichimoku, if_rendering, IRR, isrendering, melt_any, normalize, ODD,
packages_loaded, Qdate, quickbarplot, quickplot, R2, rebase, remove_caret,
remove_outliers, remove_zero, required_functions, required_packages,
rotate_clockwise, RSS, Sdate, shade_signals, show_df, showcolors, smart_round,
when_rendering
DEBUG: Processing function: bprint
DEBUG: Successfully added function: bprint
DEBUG: Processing function: check_acc
DEBUG: Successfully added function: check_acc
DEBUG: Processing function: compare_cdf
DEBUG: Successfully added function: compare_cdf
DEBUG: Processing function: desc_df
DEBUG: Successfully added function: desc_df
DEBUG: Processing function: extract_r_code
DEBUG: Successfully added function: extract_r_code
DEBUG: Processing function: find_outliers
DEBUG: Successfully added function: find_outliers
```

```
DEBUG: Processing function: quickbarplot
DEBUG: Successfully added function: quickbarplot
DEBUG: Processing function: quickplot
DEBUG: Successfully added function: quickplot
DEBUG: Processing function: remove_outliers
DEBUG: Successfully added function: remove_outliers
DEBUG: Processing function: RSS
DEBUG: Successfully added function: RSS
DEBUG: Processing function: show_df
DEBUG: Successfully added function: show_df
DEBUG: Processing function: smart_round
DEBUG: Successfully added function: smart_round
DEBUG: Total functions added: 12
Standalone R script created successfully!
Output file: climate_derivatives_just_code.R
Dependencies automatically included.
```

time of creation

```
Warning in as.POSIXlt.POSIXct(x, tz): unknown timezone '
'
```

```
[1] "2025-10-27 13:46:14 GMT"
```

LAST MODIFICATION

```
Warning in as.POSIXlt.POSIXct(x, tz): unknown timezone '
'
```

```
[1] "2025-10-27 14:14:33 GMT"
```

Last Access

```
Warning in as.POSIXlt.POSIXct(x, tz): unknown timezone '
'
```

```
[1] "2025-10-27 14:14:33 GMT"
```

Last Render

```
Warning in as.POSIXlt.POSIXct(x, tz): unknown timezone '
'
```

```
[1] "2025-10-27 14:14:47 GMT"
```

References:

1. Weather Futures and Options (CME Group Inc, 2021) [LINK]
2. Managing Climate Risk with CME Group Weather Futures and Options (Dominic Sutton-Vermeule, 20-Jan 2021) [LINK]
3. MANAGING CLIMATE RISK IN THE U.S. FINANCIAL SYSTEM, ISBN: 978-0-578-74841-2 (2020) [LINK]

2 Climate data download

Much of the weather market is dominated by temperature derivatives, aimed to protect the holder from unexpected temperature prints. The underlying used to trade these contracts is either Heating Degree Days (HDD) or Cooling Degree Days (CDD). For a specific day n , these can be defined as:

The most popular temperature instruments traded are options, whose payoff function depends on a cumulative sum over a longer period, usually an entire season:

$$HDD_n = \max(T_{ref} - T_n, 0)$$

$$CDD_n = \max(T_n - T_{ref}, 0)$$

Option Type	Protection Against		Exercise When	Payout	Example
HDD call	Overly cold winters		$HDD > K$	$\alpha \cdot (HDD - K)$	Farmers
HDD put	Overly warm winters		$HDD < K$	$\alpha \cdot (K - HDD)$	Ski resorts
CDD call	Overly hot summers		$CDD > K$	$\alpha \cdot (CDD - K)$	Utilities
CDD put	Overly cold summers		$CDD < K$	$\alpha \cdot (K - CDD)$	Beaches

NASA Prediction Of Worldwide Energy Resources (POWER) | Data Access Viewer (DAV) [LINK]

This dataset is from the Agroclimatology community, which is crucial for ensuring reproducibility in climate research.

This study utilizes temperature data obtained from the MERRA-2 (Modern-Era Retrospective analysis for Research and Applications, Version 2) dataset, which provides detailed meteorological measurements at a spatial resolution of 0.5 x 0.625 degrees latitude/longitude.

The specific region analyzed, with an average elevation of 288.19 meters, is located at a latitude of 51.5221, and longitude of -0.0903.

```
library(leaflet, quietly = TRUE, warn.conflicts = FALSE)
map_data <- data.frame(
  name = "Location",
  lat = 51.5221,
  lon = -0.0903
)
# Create a leaflet map
if (knitr::is_html_output()) {
  leaflet(map_data) %>%
    addTiles() %>% # Add default OpenStreetMap tiles
    addCircleMarkers(~lon, ~lat, radius = 5, color = "red", fillOpacity = 0.5)
  %>%
    setView(lng = map_data$lon, lat = map_data$lat, zoom = 13)
} else {
  print("interactive map of disneyworld in the html version")
}
```

```
[1] "interactive map of disneyworld in the html version"
```

It is important to note that the dataset uses a value of -999 to denote missing data, which I've removed and replaced with the mean of the overall dataset. Either when a parameter cannot be computed or falls outside the range of available source data

Parameter(s):

- T2M_MAX = Temperature at 2 Meters Max for the day (C)
- T2M_MIN = Temperature at 2 Meters Min for the day (C)
- T2M_AVG = $\frac{T_{MIN} + T_{MAX}}{2}$ Temperature at 2 Meters Average for the day (C)

Temperature at 2 meters refers to the air temperature measured 2 meters (approximately 6.5 feet) above the ground. This is the standard height for temperature measurements used in meteorology because it minimizes the effects of ground heating or cooling, providing a more accurate reflection of the ambient air temperature experienced by humans and relevant to various economic activities.

Why? Temperature at 2 meters is commonly used as the basis for weather derivatives. This is because:

- **Standardization:** Temperature at 2 meters is a standardized and widely available data point, making it reliable for use in financial contracts.

- **Relevance:** Many weather-related risks, such as heating degree days (HDD) and cooling degree days (CDD), are calculated based on temperatures at this height. These metrics are commonly used in weather derivatives to hedge against risks related to heating and cooling needs.
- **Accuracy:** Because it represents the temperature most relevant to human activities and energy consumption, it is directly applicable for creating and settling weather derivatives that are based on temperature-related indices.

```
ORIGINAL_DATASET <- read.csv("London_DATA.csv")

DATASET <- ORIGINAL_DATASET %>%
  mutate(T_MAX=remove_outliers(T2M_MAX, fill = "NA") %>% na.approx) %>%
  mutate(T_MIN=remove_outliers(T2M_MIN, fill = "NA") %>% na.approx) %>%
  mutate(DAY = as.Date(ORIGINAL_DATASET$DOY - 1, origin =
paste0(ORIGINAL_DATASET$YEAR, "-01-01"))) %>%
  mutate(Month=month(DAY)) %>%
  dplyr::select(DAY, YEAR, Month, DOY, T_MAX, T_MIN)

DATASET$T_AVG <- apply(DATASET[c("T_MAX", "T_MIN")], 1, mean)
sens_misfunctions <-
sum(ifelse(find_outliers(ORIGINAL_DATASET$T2M_MAX)==0,FALSE,TRUE))

if (sens_misfunctions == 0) {
  print("No days where the sensors malfunctioned")
} else{

ORIGINAL_DATASET[ifelse(find_outliers(ORIGINAL_DATASET$T2M_MAX)==0,FALSE,TRUE),]
%>%
  group_by(YEAR) %>%
  summarise("Days" = length(DOY)) %>%
  ggplot( aes(x = YEAR, y = Days))+
  geom_bar(stat = "identity") +
  labs(x= NULL, y =NULL, title = "Days where the sensor malfunctioned",
      subtitle = "Identified by remove outliers")
}
```

```
[1] "No days where the sensors malfunctioned"
```

```
if (knitr::is_html_output()) {
DATASET %>% select(DAY, T_MAX, T_MIN, T_AVG) %>%
  smart_round(digits = 3) %>%
  datatable() %>%
  formatStyle("T_MAX",
background = styleColorBar(range(DATASET$T_MAX), "indianred3"),
backgroundSize = "100% 80%",
```

```
backgroundRepeat = "no-repeat"
)
} else {
  print("interactive table of the data in the html version")
}
```

```
[1] "interactive table of the data in the html version"
```

3 Initial data visualization

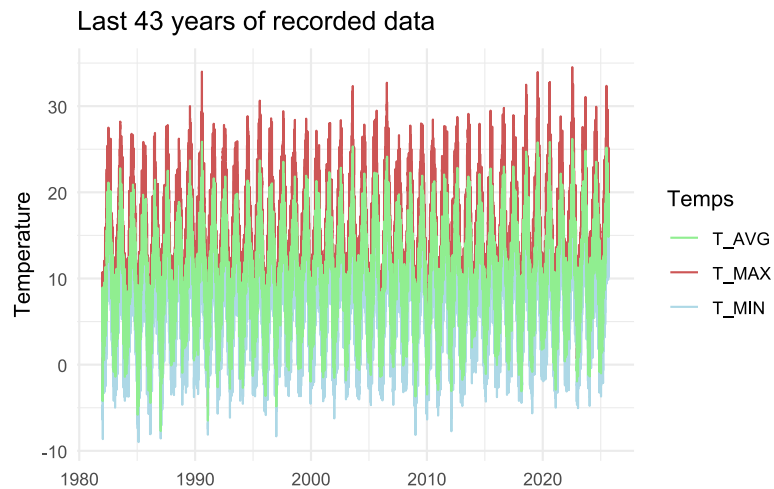
```
cleandataset <- DATASET %>%
  select(T_MAX, T_MIN, T_AVG) %>%
  xts(order.by = DATASET$DAY)

desc_df(cleandataset)
```

	n	mean	sd	median	trimmed	min	max	range	skew	kurtosis
T_MAX	15958	14.3595	6.4903	14.290	14.3596	-6.840	34.530	41.37	0.0196	-0.7345
T_MIN	15958	6.2889	4.9606	6.480	6.3782	-8.990	18.900	27.89	-0.1434	-0.6944
T_AVG	15958	10.3242	5.5871	10.325	10.3765	-7.705	26.205	33.91	-0.0570	-0.7346
	se	%NA	Q.1%	Q.25%	Q.75%	Q.99%				
T_MAX	0.0514	0	1.1300	9.36	19.5500	27.8600				
T_MIN	0.0393	0	-4.1543	2.58	10.1600	15.9343				
T_AVG	0.0442	0	-1.3122	6.06	14.8188	21.5622				

```
plot1 <- cleandataset %>%
  as.data.frame() %>%
  mutate(year = index(cleandataset)) %>%
  ggplot(aes(x=year))+
  geom_line(aes(y=T_MAX, color = "T_MAX"))+
  geom_line(aes(y=T_MIN, color = "T_MIN"))+
  geom_line(aes(y=T_AVG, color = "T_AVG"))+
  labs(title = "Last 43 years of recorded data", y="Temperature", x=NULL)+
  scale_color_manual(name = "Temps",
    values = c(T_MAX = "indianred3", T_MIN = "lightblue", T_AVG
  = "lightgreen"))

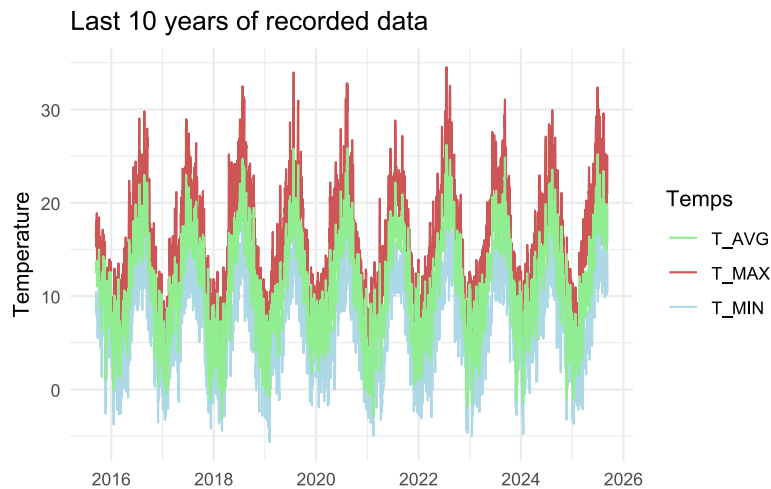
plot1
```



However this is not that useful so let me zoom in on the last third of the database (approximately 2014 onward)

```
NDAYS <- nrow(cleandataset)
lookback <- 365*10

tail(cleandataset, lookback) %>%
  as.data.frame() %>%
  mutate(year = index(tail(cleandataset, lookback))) %>%
  ggplot(aes(x = year))+
  geom_line(aes(y=T_MAX, color = "T_MAX"))+
  geom_line(aes(y=T_MIN, color = "T_MIN"))+
  geom_line(aes(y=T_AVG, color = "T_AVG"))+
  labs(title = "Last 10 years of recorded data", y="Temperature", x=NULL)+
  scale_color_manual(name = "Temps",
    values = c(T_MAX = "indianred3", T_MIN = "lightblue", T_AVG
= "lightgreen"))
```



```
library(gganimate)

months365 <- c() # initialize empty vector

for (i in month.abb) {
  months365 <- c(months365, i, rep(" ", 4))
}
monthplot <- months365[-52][[-47]][[-40]][[-33]][[-22]][[-13]][[-8]]

plot_data <- cleandataset %>%
  as.data.frame() %>%
  mutate(
    date = index(cleandataset),
    year = year(date),
    month = month(date),
    week = week(date)
  ) %>%
  group_by(year, week) %>%
  summarise(T_AVG = mean(T_AVG), .groups = 'drop') %>%
  arrange(year, week)

if (knitr::is_html_output()) {
  funny_plot <- ggplot(plot_data, aes(x = factor(week), y = T_AVG, group = year,
    color = T_AVG)) +
    geom_line(linewidth = 1.2, show.legend = FALSE) +
    coord_polar() +
    scale_x_discrete(breaks = 1:53, labels = monthplot) +
    scale_color_gradient(low = "blue", high = "red") +
    labs(subtitle = "Year: {frame_time}", x = NULL, y = NULL) +
    transition_reveal(year)

  animate(funny_plot, fps = 30, duration = 15, end_pause = 120)
```

```

} else {
  print("animated plot in html")
}

```

```

pivot_df <- DATASET %>%
  mutate(t_diff = c(NA, diff(T_AVG))) %>%
  select(Month, YEAR, t_diff) %>%
  pivot_wider(names_from = YEAR, values_from = t_diff, values_fn = median)
%>%
  mutate(Month = month.abb) %>%
  melt(id.vars = "Month", variable.name = "Year") %>%
  arrange(Year) %>%
  drop_na() %>%
  mutate(Year = as.integer(Year)+1980)

last_dec <- pivot_df %>%
  filter(Month == "Dec") %>%
  mutate(Year = Year - 1,
         Month = "last_Dec")

next_jan <- pivot_df %>%
  filter(Month == "Jan") %>%
  mutate(Year = Year + 1,
         Month = "Next_jan")

t_data <- bind_rows(pivot_df, next_jan) %>%
  mutate(Month = factor(Month, levels = c(month.abb, "next_Jan")),
         Month_number = as.numeric(Month)) %>% drop_na()

annotation <- t_data %>%
  slice_max(Year-1) %>%
  slice_max(Month_number)

temp_lines <- tibble(
  x = 12,
  y = c(1.5, 2.0),
  labels = c("1.5\u00B0C", "2.0\u00B0C")
)

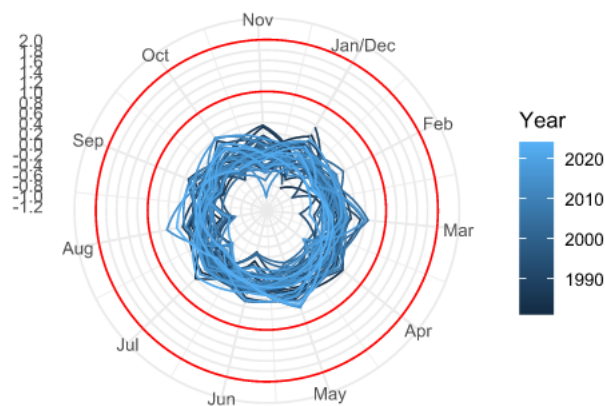
month_labels <- tibble(
  x = 1:12,
  labels = month.abb,
  y = 2.7
)

t_data %>%
  ggplot(aes(x=Month_number, y=value, group=Year, color=Year)) +

```

```
geom_hline(yintercept = c(1, 2.0), color="red") +
geom_line() +
scale_x_continuous(breaks=1:12,
                    labels=month.abb, expand = c(0,0),
                    sec.axis = dup_axis(name = NULL, labels=NULL)) +
scale_y_continuous(breaks = seq(-2, 2, 0.2),
                    limits = c(-2, 2.7), expand = c(0, -0.7),
                    sec.axis = dup_axis(name = NULL, labels=NULL)) +
coord_polar(start = 2*pi/12) +
labs(x = NULL,
     y = NULL,
     title = "Global temperature change (1981-2025)")
```

Global temperature change (1981-2025)



3.1 Overlap of the dataset

One of the points that i would need for later is the standard deviation across the days of year, a pivot table is used for this step as with this I'm able to line up all the dataset month to month, by doing this im also able to analyze seasonal patterns and look at the seasonality at a glance

```
pivot_df <- DATASET %>%
  select(DOY, YEAR, T_AVG) %>%
  pivot_wider(names_from = YEAR, values_from = T_AVG)

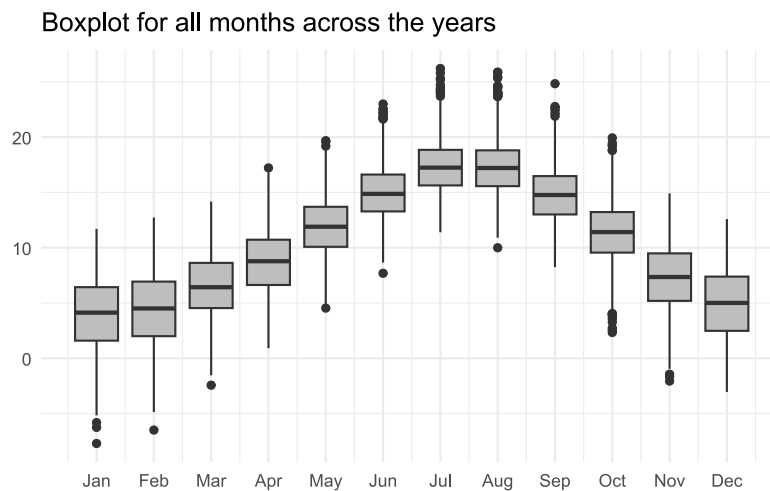
MAX_pivot_df <- DATASET %>%
  select(DOY, YEAR, T_MAX) %>%
  pivot_wider(names_from = YEAR, values_from = T_MAX)

MIN_pivot_df <- DATASET %>%
  select(DOY, YEAR, T_MIN) %>%
  pivot_wider(names_from = YEAR, values_from = T_MIN)
```

```

DATASET %>%
  select(Month, T_AVG) %>%
  group_by(Month) %>%
  ggplot(aes(x = Month, y = T_AVG, group = Month)) +
    geom_boxplot(fill = "gray") +
    scale_x_continuous(breaks = seq(1, 12, by = 1), labels = month.abb) +
    labs(title = "Boxplot for all months across the years", x = NULL, y =
  NULL)

```



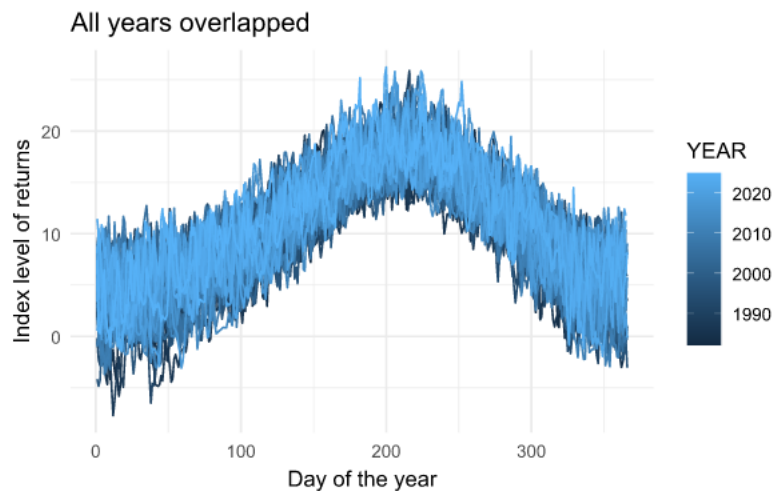
3.1.a How is this year compared to the rest

This dataset is 43 years long and there have been some really hot points and some really cold points over the years, so i wanted to look at my dataset and create a ribbon of all the max and mins, to see if this year has been so hot when looking at the averages, while this is in no way conclusive evidence it at least gives you a glimpse at how the climate is changing especially when looking at the middle of the year.

```

ggplot(DATASET, aes(x = DOY, y = T_AVG, group = YEAR, color = YEAR)) +
  geom_line() +
  labs(title = "All years overlapped", x = "Day of the year", y = "Index level
of returns")

```



```
THISYEAR <- data.frame(
  c(DATASET[DATASET$YEAR == 2025, ]["T_MAX"]),
  c(DATASET[DATASET$YEAR == 2025, ]["T_AVG"]),
  c(DATASET[DATASET$YEAR == 2025, ]["T_MIN"])
)

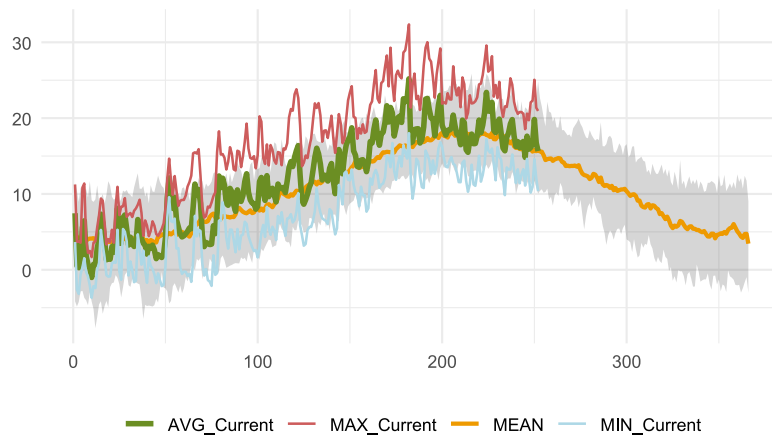
cbind(pivot_df,
      "MIN" = apply(pivot_df[-1], 1, min, na.rm = TRUE),
      "MAX" = apply(pivot_df[-1], 1, max, na.rm = TRUE),
      "MEAN" = apply(pivot_df[-1], 1, mean, na.rm = TRUE)) %>%
  round(2) %>%
  ggplot(aes(x = DOY)) +
    geom_ribbon(aes(ymin = MIN, ymax = MAX), alpha = 0.2) +
    geom_line(aes(y = MEAN, color = "MEAN"), linewidth = 1) +
    geom_line(aes(y = pivot_df$`2025`, color = "AVG_Current"), linewidth =
1.3) +
    geom_line(aes(y = MAX_pivot_df$`2025`, color = "MAX_Current"), linewidth =
0.6, alpha = 1) +
    geom_line(aes(y = MIN_pivot_df$`2025`, color = "MIN_Current"), linewidth =
0.6, alpha = 1) +
    scale_color_manual(name = NULL, values = c(MEAN = "orange2", AVG_Current =
"olivedrab",
                                                MIN_Current = "lightblue",
                                                MAX_Current = "indianred")) +
    labs(title = "Is this year hotter on average?", y = NULL, x = NULL)+
    theme(legend.position = "bottom")
```

Warning: Removed 114 rows containing missing values or values outside the scale range (``geom_line()``).

Removed 114 rows containing missing values or values outside the scale range

```
(`geom_line()`).
Removed 114 rows containing missing values or values outside the scale range
(`geom_line()`).
```

Is this year hotter on average?



3.2 Seasonal analysis

Now i can look at the distribution of my database, first i need to distinguish the data between summer periods and winter periods by taking the day of the year and splitting it so that

Winter ~ October-March

Summer ~ April-September

```
DATASET_seas <- DATASET %>%
  group_by(Month < 4 | Month > 9) %>%
  rename("Season" = "Month < 4 | Month > 9") %>%
  mutate(Season = ifelse(Season, "Winter", "Summer"))

DATASET_seas[-1] %>%
  xts(order.by = DATASET$DAY) %>%
  show_df() %>%
  gt() %>%
  tab_header(title = "Temperature Overview") %>%
  opt_stylize(style = 5, add_row_stripping = TRUE) %>%
  cols_align(aligned = "center") %>%
  sub_missing(columns = everything(), missing_text = ":")
```

```
Warning: There was 1 warning in `mutate()`.
i In argument: `across(where(is.numeric), round, digits = digits)`.
Caused by warning:
! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
```

Supply arguments directly to `\.fns`` through an anonymous function instead.

```
# Previously
across(a:b, mean, na.rm = TRUE)

# Now
across(a:b, \(x) mean(x, na.rm = TRUE))
```

Temperature Overview

DATE	YEAR	Month	DOY	T_MAX	T_MIN	T_AVG	Season
1982-01-01	1982	1	1	4.16	1.13	2.645	Winter
1982-01-02	1982	1	2	10.53	0.94	5.735	Winter
1982-01-03	1982	1	3	9.81	7.45	8.630	Winter
1982-01-04	1982	1	4	10.75	7.17	8.960	Winter
1982-01-05	1982	1	5	9.12	1.16	5.140	Winter
:	:	:	:	:	:	:	:
2025-09-05	2025	9	248	22.08	10.17	16.125	Summer
2025-09-06	2025	9	249	22.29	10.93	16.610	Summer
2025-09-07	2025	9	250	25.06	14.59	19.825	Summer
2025-09-08	2025	9	251	21.37	12.09	16.730	Summer
2025-09-09	2025	9	252	20.95	10.37	15.660	Summer

```
winter_dataset <- DATASET_seas[DATASET_seas$Season=="Winter",]
summer_dataset <- DATASET_seas[DATASET_seas$Season=="Summer",]

cat(paste0("The difference in number of rows is approximately: ",
round(nrow(summer_dataset) / nrow(winter_dataset) - 1, 4)*100,"%", ", or ",
round((nrow(summer_dataset) / nrow(winter_dataset) - 1)*NDAYS, 1), " days"))
```

The difference in number of rows is approximately: 1.31%, or 209.4 days

```
grid.arrange(ncol=2,
ggplot(winter_dataset)+
  geom_histogram(aes(x = T_MAX, fill = "T_MAX"), alpha=0.8, bins = 80)+
  geom_histogram(aes(x = T_MIN, fill = "T_MIN"), alpha=0.8, bins = 80)+
  geom_histogram(aes(x = T_AVG, fill = "T_AVG"), alpha=0.8, bins = 80)+
  labs(title = "Distribution charts of winter",x=NULL)+
  scale_fill_manual(name = NULL,
```

```

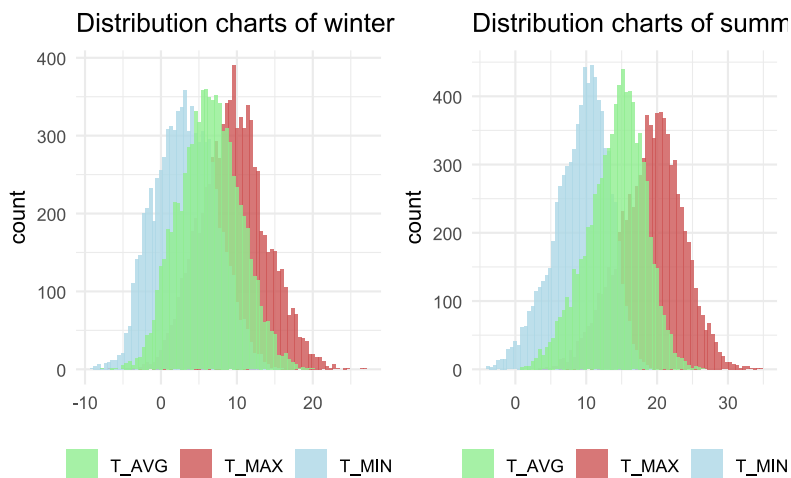
        values = c(T_MAX = "indianred3",T_MIN = "lightblue",T_AVG
= "lightgreen"))+
    theme(legend.position = "bottom")

,

ggplot(summer_dataset)+
  geom_histogram(aes(x = T_MAX, fill = "T_MAX"), alpha=0.8, bins = 80)+
  geom_histogram(aes(x = T_MIN, fill = "T_MIN"), alpha=0.8, bins = 80)+
  geom_histogram(aes(x = T_AVG, fill = "T_AVG"), alpha=0.8, bins = 80)+
  labs(title = "Distribution charts of summer",x=NULL)+
  scale_fill_manual(name = NULL,
        values = c(T_MAX = "indianred3",T_MIN = "lightblue",T_AVG
= "lightgreen"))+
  theme(legend.position = "bottom")

)

```



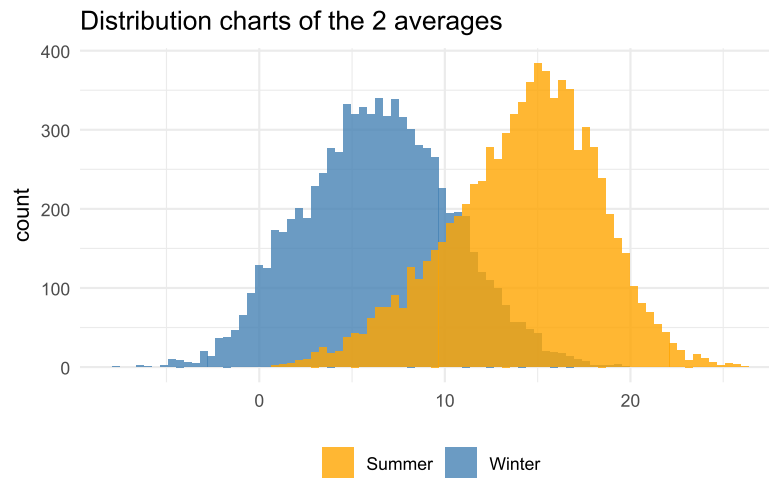
Its with this choice of season im still able to keep the data very balanced. clear to see that there
 Lastly I'm able to plot just the two average temperatures of winter and of summer for the average temperatures

```

ggplot()+
  geom_histogram(data = winter_dataset, aes(x = T_AVG, fill = "Winter"),
alpha=0.8, bins = 80)+
  geom_histogram(data = summer_dataset, aes(x = T_AVG, fill = "Summer"),
alpha=0.8, bins = 80)+
  labs(title = "Distribution charts of the 2 averages",x=NULL)+
  scale_fill_manual(name = NULL,

```

```
values = c(Winter="steelblue", Summer="orange"))+
theme(legend.position = "bottom")
```



Just to understand the records of my dataset i wanted to see what months on record had the hottest and coldest days across all dataset

```
DATASET_seas %>%
  group_by(Month) %>%
  summarise(T_min_min = min(T_MIN),
            T_min_max = max(T_MIN),
            T_max_min = min(T_MAX),
            T_max_max = max(T_MAX)) %>%
  round(3) %>%
  mutate(Month = month.abb) %>%
  gt() %>%
  opt_stylize(5) %>%
  tab_header("Data exploration", "what is the highest and lowest in my
database") %>%
  cols_label(T_min_min = "Min",
            T_min_max = "Max",
            T_max_min = "Min",
            T_max_max = "Max") %>%
  tab_spanner(label = "T_MIN", columns = 2:3) %>%
  tab_spanner(label = "T_MAX", columns = 4:5)
```

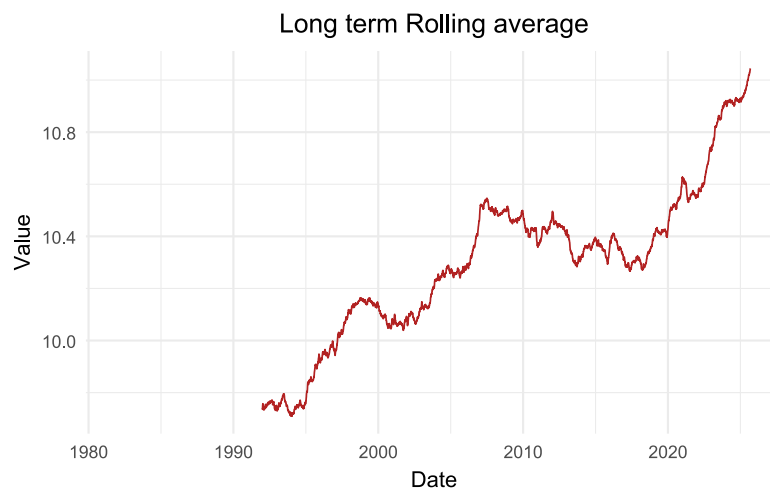
Data exploration

what is the highest and lowest in my database

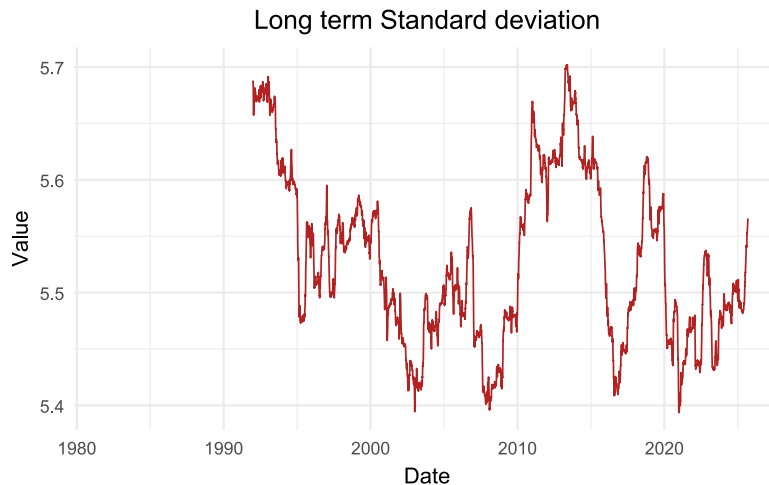
Month	T_MIN		T_MAX	
	Min	Max	Min	Max
Jan	-8.64	10.73	-6.84	13.14
Feb	-8.99	10.72	-4.85	16.88
Mar	-5.24	9.97	-0.43	20.44
Apr	-4.11	11.65	3.54	25.19
May	-0.66	14.17	7.49	26.47
Jun	1.68	17.27	11.79	30.42
Jul	5.60	18.86	15.10	34.53
Aug	5.48	18.90	13.15	34.01
Sep	2.52	18.59	10.67	31.07
Oct	-2.69	16.00	6.62	27.07
Nov	-4.77	12.66	-0.04	17.67
Dec	-5.91	11.45	-1.55	13.94

3.3 Long term trends

```
quickplot(SMA(DATASET$T_AVG, lookback), show_legend = F, title = "Long term  
Rolling average",  
x_start = min(DATASET$YEAR), x_step = 1 / 365.25)
```



```
quickplot(
  runSD(DATASET$T_AVG, lookback), show_legend = F, title = "Long term Standard
  deviation",
  x_start = min(DATASET$YEAR), x_step = 1 / 365.25)
```



4 Seasonal decomposition

The first step in pricing temperature options is to decompose the time series into distinct components that each represent different underlying patterns. This process is essential for understanding the structure of the data and for developing predictive models. In basic terms it can be expressed as the sum of three main components: $y_t = T_t + S_t + e_t$. Here, T_t is the trend component that captures the long-term movement, S_t represents the seasonal variations (periodic patterns), and e_t refers to the residuals or noise in the data.

which in our case becomes

$$\overline{T}_t = T_{trend} + T_{seasonal} + Residuals$$

The approach was to develop each component independently

$$T_{trend} = a + b \cdot t$$

:

The trend component could be as simple as a straight line or a moving average

This form captures the cyclical nature of temperature changes throughout the year, like daily or annual temperature fluctuations.

$$T_{seasonal} = \alpha \cdot \sin(\omega \cdot t + \theta)$$

- α represents the amplitude
- ω is the angular frequency (related to the period of the cycle)

- θ is the phase shift

And lastly the residuals are just:

$$Residuals = T_t - T_{trend} + T_{seasonal}$$

Recent research has suggested that partial Fourier decomposition can be effective in modeling temperature data, particularly by omitting the cosine component. This approach simplifies the seasonal model while still accurately representing the cyclical nature of temperature changes. The sinusoidal term is often sufficient to describe seasonal temperature variations, making the model both interpretable and computationally efficient.

Before this I used a de-noising procedure using Gaussian Convolution Filter as it also enhances the effectiveness of the Fourier transformation by reducing noise, allowing you to identify the dominant frequencies or cycles in your temperature data more accurately.

- **Gaussian Convolution Filter:**

- A **convolution** is a mathematical operation used to blend or smooth data. It involves applying a kernel (a small array of weights) across the data to compute a weighted average. The Gaussian kernel is a specific type of convolution filter that assigns weights based on the Gaussian (normal) distribution, which is a bell-shaped curve.
- The **Gaussian kernel** is widely used for smoothing because it gives more weight to the central points (closer to the middle) while gradually reducing the weights for points further away. This is mathematically expressed as:

$$G(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \cdot e^{-\frac{x^2}{2\sigma^2}}$$

- **Moving Window:** The Gaussian kernel slides across each point in the time series. For each position, it computes a weighted sum where the weights are defined by the Gaussian kernel.
- **Local Averaging:** The computed value represents a smoothed average of the points within the window's range. Points closer to the center of the window contribute more heavily to the average than those farther away, as per the Gaussian weights.
- **Symmetric Filtering:** using 2 sides makes this operation symmetric, applying the kernel both forward and backward across the data, ensuring the filtering effect is centered.
- **Conclusion and data inspection:** in visual terms the de-noised data should look smoother with slightly lower peaks

```
temps <- DATASET
apply_convolution <- function(x, kernel) {
  # Use filter() from stats package to apply convolution
  filtered <- stats::filter(x, kernel, sides = 2) # Use sides = 2 for
symmetric filter
  return(filtered)
```

```

}

kernel <- dnorm(-3:3)
data.frame("Gaussian_Kernel" = round(kernel, 10))

```

```

      Gaussian_Kernel
1      0.004431848
2      0.053990967
3      0.241970725
4      0.398942280
5      0.241970725
6      0.053990967
7      0.004431848

```

```

temps$Denoised <- apply_convolution(temps$T_AVG, kernel)
temps$Denoised <- na.fill(temps$Denoised, mean(temps$Denoised, na.rm = TRUE))

temps$Trend <- SMA(temps$Denoised, n = lookback)

library(nlme, quietly = T, warn.conflicts = F)

# Define the model
sin_component <- function(t, a, b, alpha, theta) {
  omega <- 2 * pi / 365.25
  a + b * t + alpha * sin(omega * t + theta)
}
omega <- 2 * pi / 365.25

# Fit model using non linear squares
temps$NUM_DAY <- 1:nrow(temps)

fit <- nls(Denoised ~ sin_component(NUM_DAY, a, b, alpha, theta),
  data = temps,
  start = list(a = 1, b = 0, alpha = 1, theta = 0))

# Get coefficients and confidence intervals for the model
MOD_params <- coef(fit)
confint_fit <- suppressMessages(confint(fit))

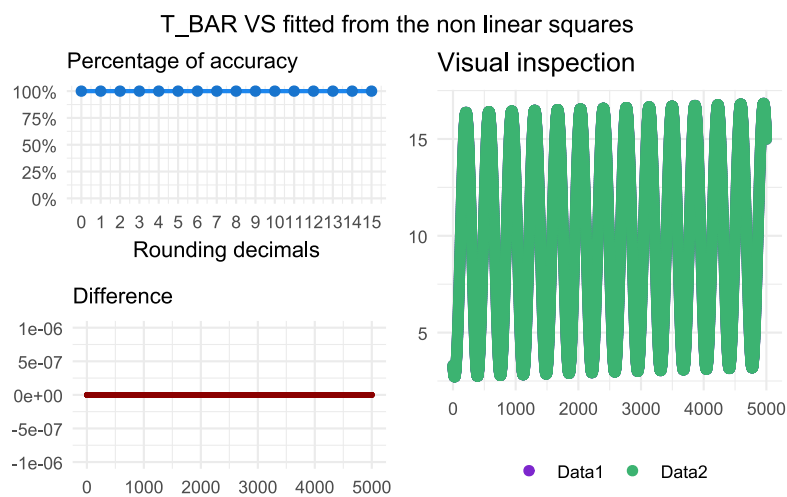
temps$SEAS <- MOD_params["alpha"] * sin(omega * temps$NUM_DAY +
MOD_params["theta"])
temps$TREND <- MOD_params["a"] + MOD_params["b"] * temps$NUM_DAY
temps$BAR <- temps$TREND + temps$SEAS
temps$RESID <- temps$T_AVG - temps$TREND - temps$SEAS

```

just to be certain i looked at if the residuals and the fitted values matched the $\overline{T}$ and *Residuals* by checking percentage of same values as the rounding decimals increase The logic is that if the data sets are anything alike (like 2 different models results for the same dataset) you should see how far in the rounding are the data sets similar

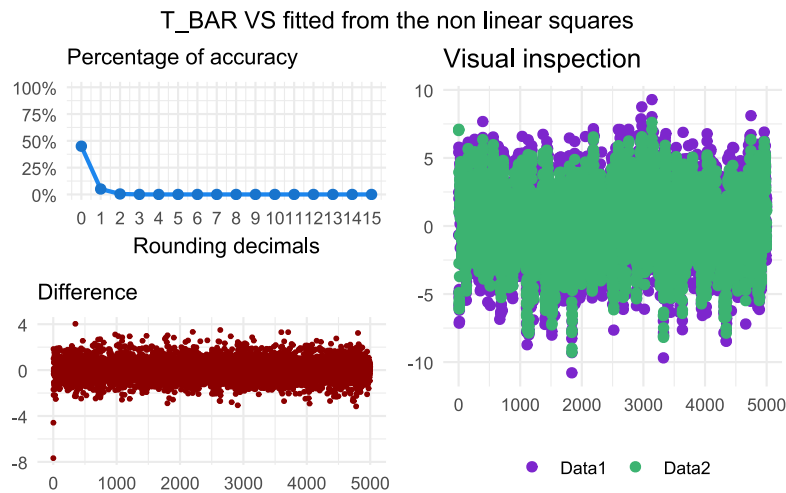
```
check_acc(temps$BAR, fitted(fit),15, title = "T_BAR VS fitted from the non
linear squares")
```

Scale for y is already present.
Adding another scale for y, which will replace the existing scale.



```
check_acc(temps$RESID, residuals(fit),15, title = "T_BAR VS fitted from the
non linear squares")
```

Scale for y is already present.
Adding another scale for y, which will replace the existing scale.



4.1 Model performance

for assessing my model performance I'm looking at the values and confidence intervals and looking at the Residual sum of squares and the mean absolute error

- The **RSS function** calculates the Residual Sum of Squares, which is a measure of the discrepancy between the observed data and the model's predictions. A lower RSS indicates a better fit of the model to the data, meaning that the model's predictions are closer to the actual observed values.
- The **MAE function** is the average of the absolute differences between the observed and predicted values. It gives an idea of how far, on average, the predictions are from the actual values, without considering the direction of the error. A lower MAE indicates that the model's predictions are generally close to the observed data.

```
a : 9.533 CI ~normally [ 9.458 , 9.608 ]
b : 0 CI ~normally [ 0 , 0 ]
alpha : -6.814 CI ~normally [ -6.867 , -6.761 ]
theta : -5.147 CI ~normally [ -5.155 , -5.14 ]
```

RSS model sine curve: 121777.4

MAE model fit: 2.23

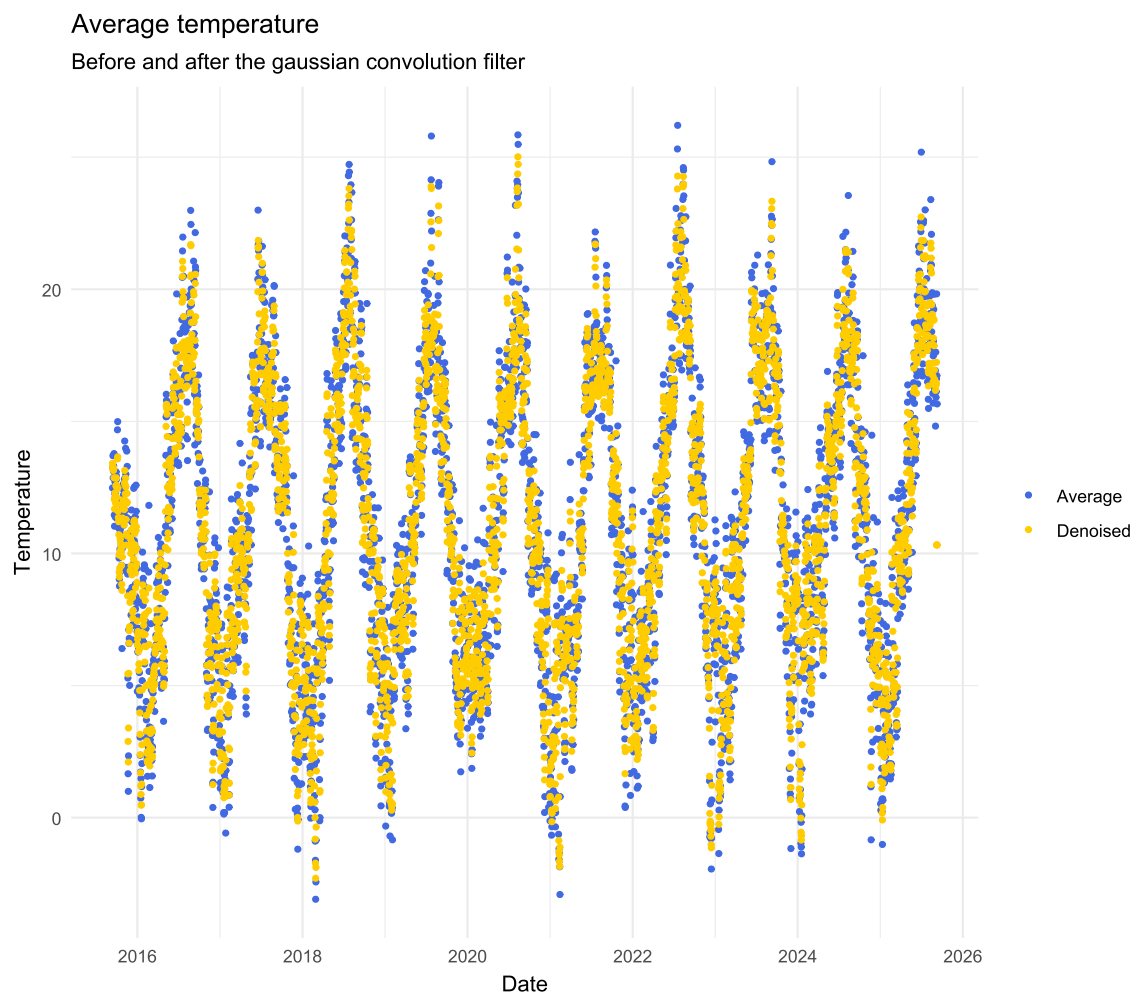
4.2 Visualization of results

```
# plot denoised
ggplot(tail(temps, lookback), aes(x=DAY))+
  geom_point(aes(y = T_AVG, color = "Average"), size = 1)+
  geom_point(aes(y = Denoised, color = "Denoised"), size = 1)+
```

```

scale_color_manual(name=NULL, values = c(Average = "royalblue", Denoised =
"#ffcc00"))+
labs(title = "Average temperature", x = "Date", y = "Temperature",
      subtitle = "Before and after the gaussian convolution filter")

```



```

temps_xts <- temps %>%
  select(T_AVG, Denoised, TREND, SEAS, RESID) %>%
  xts(order.by = temps$DAY) %>%
  tail(lookback)

# Plot seasonal decomposition from Avg to residuals
grid.arrange(nrow=5, top = paste0("Classical decomposition - last ",
lookback/365, " years"),

  temps_xts$T_AVG %>%
    quickplot(subtitle = "Average Temperature", show_legend = F, xlab = NULL,

```

```

ylab = "Temps",
  type = geom_point, show_x = F),

temps_xts$Denoised %>%
  quickplot(subtitle = "Denoised", show_legend = F, xlab = NULL, ylab =
"Temps",
  type = geom_point, show_x = F),

temps_xts$TREND %>%
  quickplot(subtitle = "Trend", show_legend = F, xlab = NULL, ylab =
"Temps" , show_x = F),

temps_xts$SEAS %>%
  quickplot(subtitle = "Seasonal", show_legend = F, xlab = NULL, ylab =
"Temps" , show_x = F),

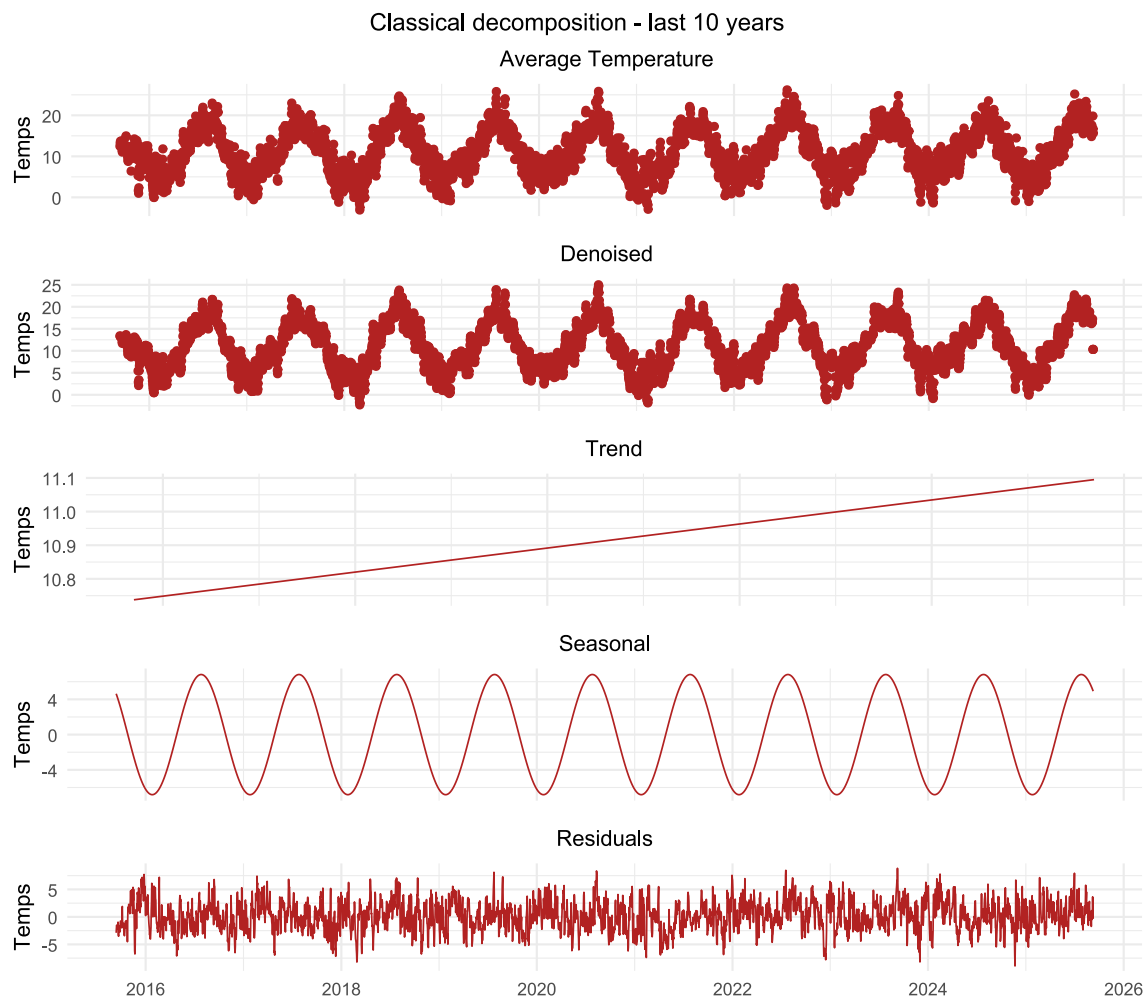
temps_xts$RESID %>%
  quickplot(subtitle = "Residuals", show_legend = F, xlab = NULL, ylab =
"Temps"))

```

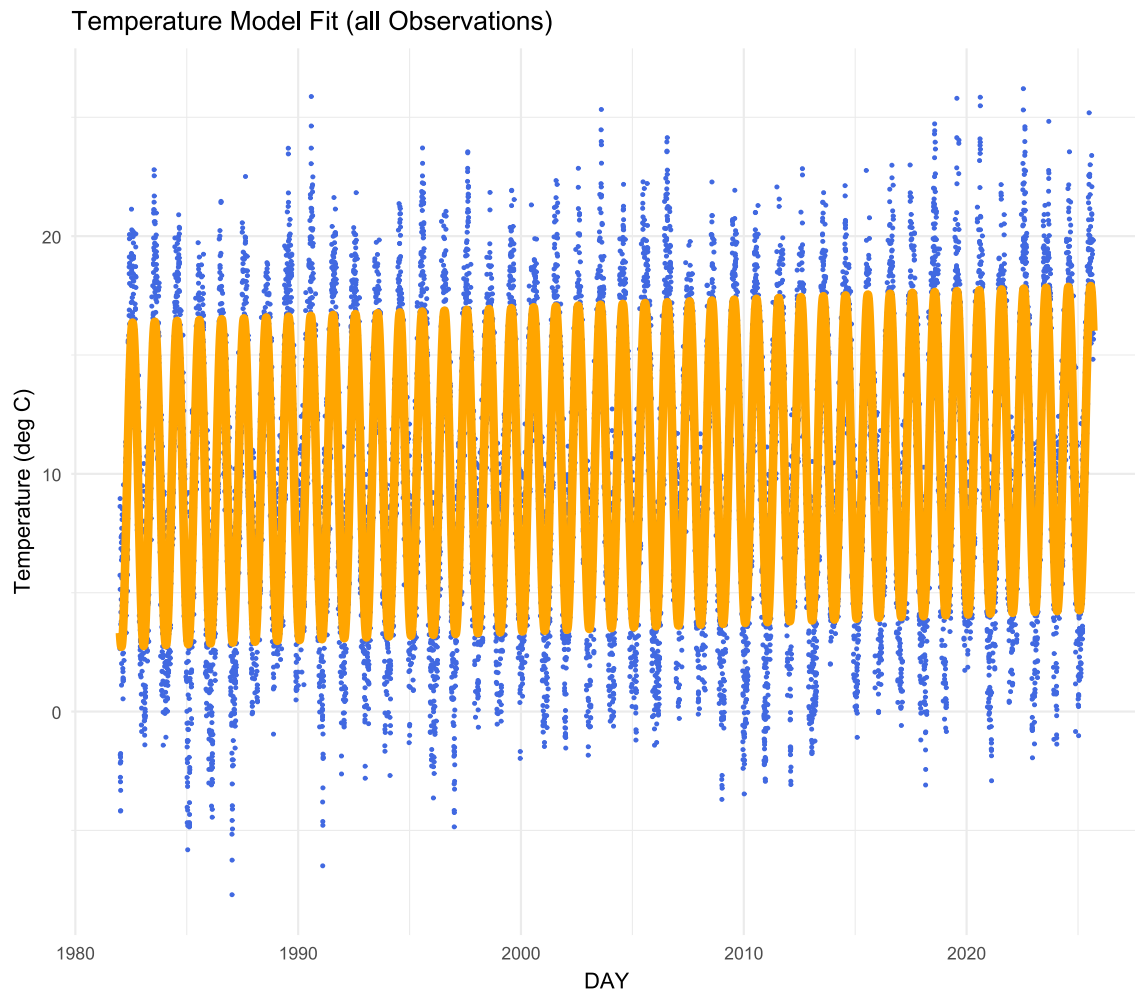
```

Warning in type(linewidth = linewidth, alpha = alpha): Ignoring unknown
parameters: `linewidth`
Ignoring unknown parameters: `linewidth`

```



```
# Plot original vs. fitted data
ggplot(temps, aes(x = DAY)) +
  geom_point(aes(y = T_AVG), color = 'royalblue', size = 0.5) +
  geom_line(aes(y = BAR), color = 'orange', linewidth=2) +
  labs(title = "Temperature Model Fit (all Observations)", y = "Temperature
(deg C)")
```



4.2.a Check for possible model degradation

Checking for model degradation across time by looking at the first and last 10 years to see if there is a meaningful shift in where the sine curve and the average temperature

```
grid.arrange(nrow = 2, ncol = 2,
ggplot(temps %>% head(lookback), aes(x = DAY)) +
  geom_point(aes(y = T_AVG), color = 'royalblue', size = 0.5) +
  geom_line(aes(y = BAR), color = 'orange', linewidth=2) +
  labs(title = paste0("Temperature Model Fit (First ", lookback/365, "
years)"),x=NULL, y = NULL),

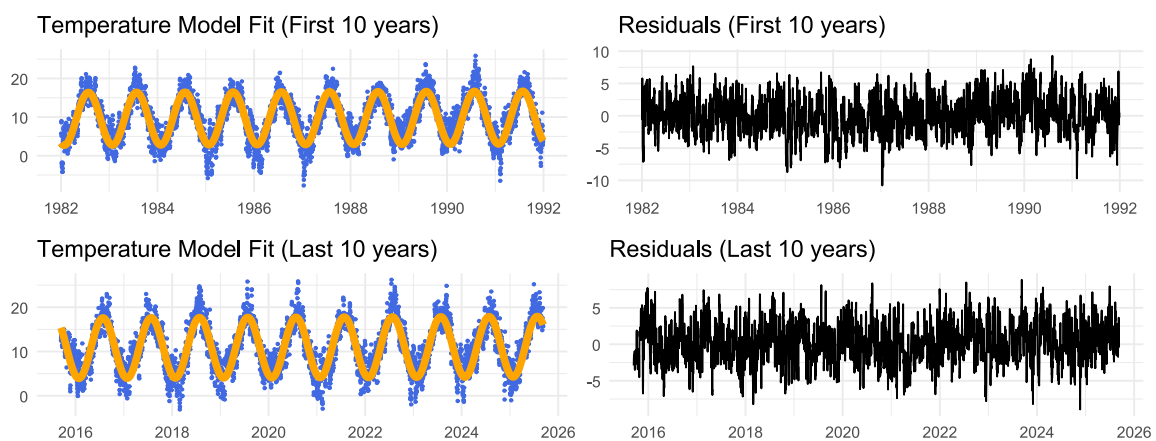
ggplot(temps %>% head(lookback), aes(x = DAY)) +
  geom_line(aes(y = RESID), color = 'black', linewidth=0.5) +
  labs(title = paste0("Residuals (First ", lookback/365, " years)"),x=NULL, y
= NULL),
```

```

ggplot(temps %>% tail(lookback), aes(x = DAY)) +
  geom_point(aes(y = T_AVG), color = 'royalblue', size = 0.5) +
  geom_line(aes(y = BAR), color = 'orange', linewidth=2) +
  labs(title = paste0("Temperature Model Fit (Last ", lookback/365, "
years)"), x=NULL, y = NULL),

ggplot(temps %>% tail(lookback), aes(x = DAY)) +
  geom_line(aes(y = RESID), color = 'black', linewidth=0.5) +
  labs(title = paste0("Residuals (Last ", lookback/365, " years)"), x=NULL, y =
NULL)
)

```



4.3 Residuals analysis and diagnostics

1. Autocorrelation Function (ACF) Plot of Residuals:

The ACF plot checks for any correlation between residuals at different lags. If residuals are uncorrelated (i.e., resemble white noise), all values should fall within the confidence bands, indicating no significant autocorrelation.

2. Partial Autocorrelation Function (PACF) Plot of Residuals:

The PACF plot displays the partial correlation of residuals with their own lagged values, considering the effect of any intermediate lags. Like the ACF plot, if the model fits well, the PACF values should also fall within the confidence bands, showing no significant partial autocorrelations.

3. Normality Check using QQ Plot:

The QQ plot assesses whether the residuals are normally distributed. Points should lie along the reference line; significant deviations from this line suggest that the residuals do not follow a normal distribution, which could imply model misspecification or the presence of outliers.

4. Histogram of Residuals with Normal Curve:

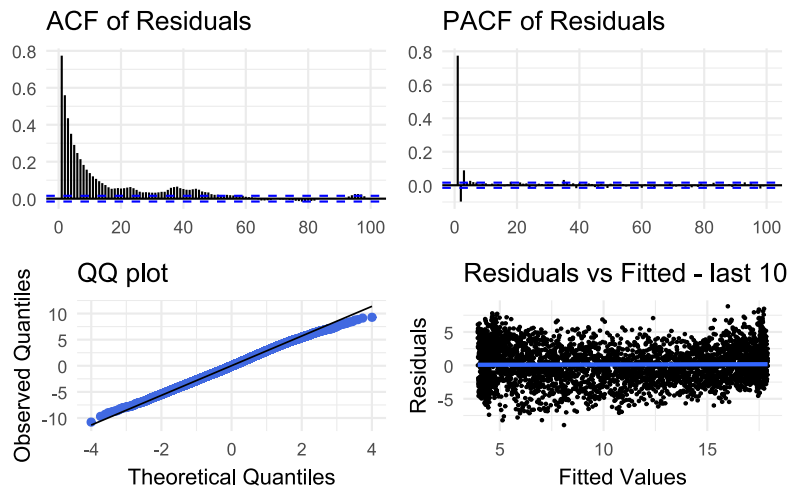
The histogram visualizes the distribution of residuals, and the overlaid normal curve helps assess normality. If the histogram closely follows the bell-shaped curve, it suggests normal residuals. The vertical line representing kurtosis indicates whether the residuals are more or less peaked than a normal distribution.

5. Residuals vs. Fitted Values Plot:

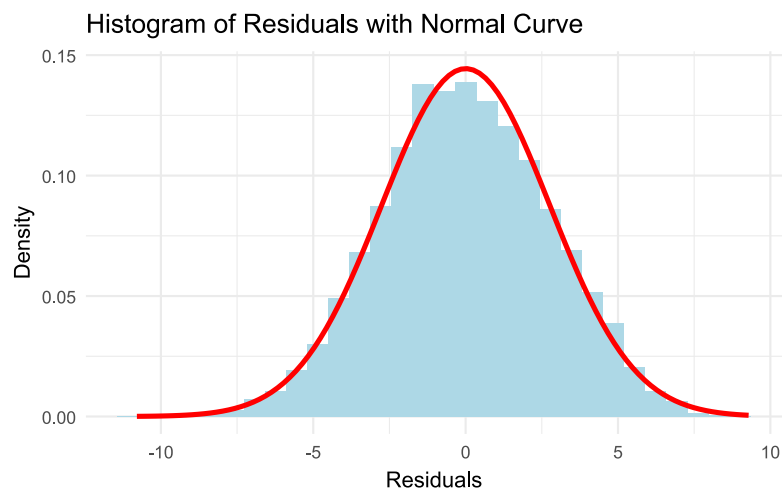
The plot checks for patterns in the residuals against fitted values. Ideally, the residuals should be randomly scattered around zero, indicating homoscedasticity (constant variance). Any visible pattern or trend suggests issues like non-linearity, heteroscedasticity, or omitted variables.

```
grid.arrange(nrow = 2,  
# ACF  
ggAcf(temps$RESID, lag.max = 100)+  
  labs(title = "ACF of Residuals", x = NULL, y = NULL),  
  
# PACF  
ggPacf(temps$RESID, lag.max = 100)+  
  labs(title = "PACF of Residuals", x = NULL, y = NULL),  
  
## Check normality of residuals using QQ plot  
ggplot(temps, aes(sample = RESID)) +  
  stat_qq(color="royalblue")+  
  stat_qq_line(color = "black", linewidth = 0.4)+  
  labs(title = "QQ plot", x="Theoretical Quantiles", y= "Observed Quantiles"),  
  
## Check for heteroskedasticity or any pattern in residuals  
ggplot(temps %>% tail(lookback), aes(x=BAR, y = RESID))+  
  geom_point(size = 0.4)+  
  geom_smooth(method = "lm")+  
  labs(title = paste0("Residuals vs Fitted - last ", lookback/365, " years"),  
x= "Fitted Values", y = "Residuals"))
```

```
`geom_smooth()` using formula = 'y ~ x'
```



```
# Histogram with bell curve and kurtosis
ggplot(temps, aes(x = RESID)) +
  geom_histogram(aes(y = after_stat(density)), fill = "lightblue", bins = 30)
+
  stat_function(fun = dnorm, args = list(mean = mean(temps$RESID), sd =
sd(temps$RESID)),
               color = "red", linewidth = 1.2) +
  # geom_vline(xintercept = skewness(temps$RESID)[1], linetype = "dashed",
  linewidth=1, color = "darkred")+
  labs(title = "Histogram of Residuals with Normal Curve", x = "Residuals", y
= "Density",
       # subtitle = "Vertical line is kurtosis"
       )
```



5 Ornstein-Uhlenbeck (OU) process

Temperature dynamics using a mean-reverting stochastic process like the Ornstein-Uhlenbeck (OU) process, incorporate the seasonal adjustment into the drift rate to ensure that the expected value of the temperature follows the seasonal mean temperature.

The Ornstein-Uhlenbeck (OU) process is a type of stochastic (random) process that is often used in physics, finance, and other fields to model systems that exhibit some form of “mean-reverting” behavior. Let’s start from the basics to understand what this means.

A stochastic process is a collection of random variables representing the evolution of a system over time. In simpler terms, it describes how something changes when randomness is involved. For example, the price of a stock or the position of a particle under the influence of random forces can be modeled as a stochastic process.

$$dX_t = \kappa(\mu - X_t) dt + \sigma \cdot dW_t$$

- X_t is the value of the process at time t .
- μ is the long-term mean or equilibrium level toward which the process reverts.
- $\theta > 0$ is the rate of mean reversion, determining how fast the process reverts to the mean μ .
- $\sigma > 0$ is the volatility or the intensity of the random fluctuations.
- dW_t is an infinitesimal increment of a Wiener process (Brownian motion).

Mean Reversion Term: $\kappa(\mu - X_t) dt$: Representing the tendency of the process to revert to the mean μ . If X_t is above the mean μ , this term will be negative (pulling it down toward the mean). If X_t is below μ , the term will be positive (pushing it up toward the mean).

The speed of this pull or push is determined by κ . A higher κ means the process will revert to the mean faster.

Random Fluctuation Term $\sigma \cdot dW_t$: represents the random noise or shock. This is a source of randomness, and it causes the process to deviate randomly over time.

The size of the noise is controlled by σ , which is the volatility parameter.

```
temps_OU <- temps
# Define parameters for the OU process
kappa <- 1-arma(temps_OU$RESID, order = c(1,0,0))$coef[1] # Mean-reversion
rate
sigma <- sd(temps$RESID, na.rm = TRUE) # Volatility of
the process
dt <- 1 # Time step (daily
data)

cat("Kappa is estimated as:", round(kappa,4))
```

```
Kappa is estimated as: 0.2261
```

```

# Initialize variables for simulation
n <- nrow(temps_OU)                # Number of time points
T_simulated <- numeric(n)          # Simulated temperature vector
T_simulated[1] <- temps_OU$Denoised[1] # Set initial temperature to the
first observed value

# Simulate the seasonal mean as a time-varying mean (trend + seasonal
component)
T_bar <- temps_OU$BAR

# Simulate the modified OU process
for (i in 2:n) {
  # Rate of change of the seasonal mean
  dT_bar_dt <- (T_bar[i] - T_bar[i - 1]) / dt

  # Brownian motion increment
  dWt <- rnorm(1, mean = 0, sd = sqrt(dt))

  T_simulated[i] <- T_simulated[i - 1] +
    (dT_bar_dt + kappa * (T_bar[i] - T_simulated[i - 1])) * dt
  +
    sigma * dWt
}

temps_OU$OU <- T_simulated

DATE <- tail(temps$DAY, lookback)

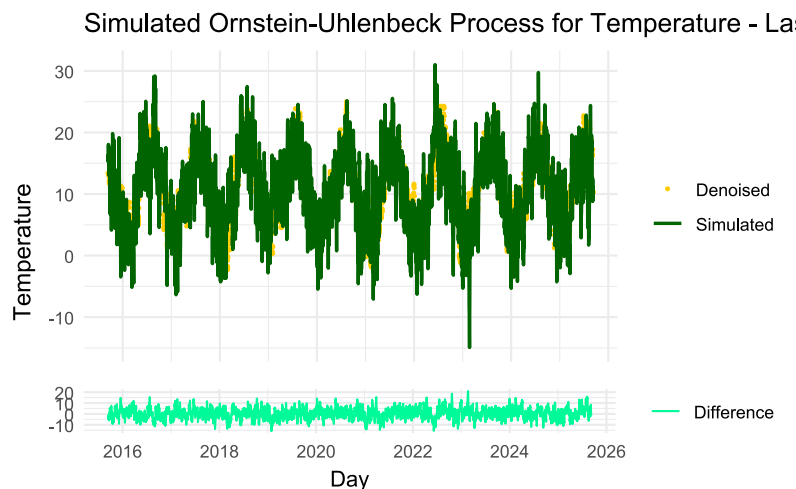
grid.arrange(layout_matrix = matrix(data = c(1,1,1,2), ncol = 1), left =
"Temperature",
temps_OU %>%
  select(Denoised, OU, BAR) %>%
  tail(lookback) %>%
  ggplot(aes(x = DATE)) +
  geom_point(aes(y = Denoised, color = "Denoised"), size = 0.5) +
  geom_line(aes(y = OU, color = "Simulated"), linewidth = 0.8) +
  scale_color_manual(name = NULL, values = c(Denoised = "#ffcc00", Simulated =
"darkgreen")) +
  labs(title = "Simulated Ornstein-Uhlenbeck Process for Temperature - Last 10
years", x = NULL, y = NULL)+
  theme(axis)+
  theme(axis.title.x = element_blank(),
        axis.text.x = element_blank(),
        axis.ticks.x = element_blank())
,
temps_OU %>%
  select(Denoised, OU, BAR) %>%
  tail(lookback) %>%

```

```

ggplot(aes(x = DATE)) +
  geom_line(aes(y = Denoised-OU, color = "Difference"), linewidth = 0.5) +
  scale_color_manual(name = NULL, values = c(Difference =
"mediumspringgreen")) +
  labs(x = "Day", y = NULL)
)

```



6 Modeling volatility

Now that i have the deterministic part of the model complete I need to model the volatility the first step was looking if there is any relation between day-of-year and volatility across the days of the year to try and model.

```

# Create a dataframe with temperature and date components
temp_t <- data.frame(
  Date = temps$DAY, # Assuming the index is a proper Date object
  T = temps$Denoised,
  day = yday(temps$DAY),
  month = month(temps$DAY)
)

vol_df <- temps %>%
  select(DAY, T_AVG) %>%
  mutate(day = yday(DAY),
         month = month(DAY)) %>%
  group_by(day) %>%
  summarise(mean = mean(T_AVG, na.rm = TRUE),
            std = sd(T_AVG, na.rm = TRUE))

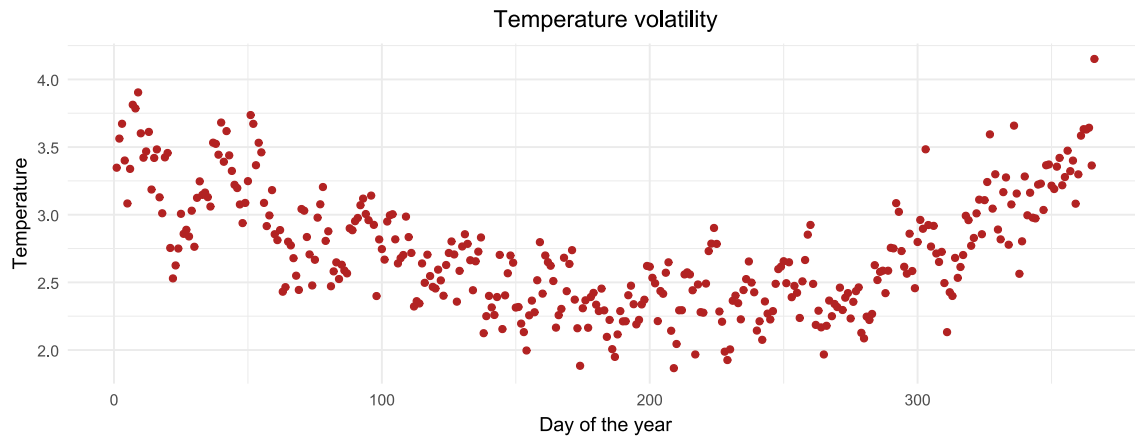
# just a shorthand for reference
x <- 1:366

```

```
y <- vol_df$std
```

```
quickplot(vol_df$std, type = geom_point, show_legend = F, xlab = "Day of the  
year", ylab = "Temperature", title = "Temperature volatility")
```

```
Warning in type(linewidth = linewidth, alpha = alpha): Ignoring unknown  
parameters: `linewidth`
```



The volatility definitely has a shape and so we can try and model it with a deterministic model. There are many models to try and use, I'm trying a bunch to be thorough:

- Polynomial regression
- Fourier transformation
- B-splines
- Kernel smoothing
- Loess modeling
- Generalized additive modelling

A good way is to look at both fit and complexity-adjusted metrics: - Akaike Information Criterion (AIC): Helps select the best model by balancing goodness-of-fit and complexity by including the number of parameters alongside the log-likelihood in its calculation. - The residual sum of squares (RSS) measures the level of variance in the error term, or residuals, of a regression model. The smaller the residual sum of squares, the better your model fits your data; the greater the residual sum of squares, the poorer your model fits your data. - Visual inspection: sometimes a model with slightly worse AIC is much smoother/interpretable.

6.1 Polynomial Regression:

Polynomial regression is an extension of linear regression where the relationship between the independent variable(s) and the dependent variable is modeled as an $n - th$ degree polynomial. Polynomial regression allows for curvature by adding higher-order terms:

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \dots + \beta_d x^d + \epsilon$$

where d is the degree of the polynomial.

Temperature volatility (or variance in fluctuations) across the year often follows seasonal patterns: higher in transition months (spring/fall), lower in stable periods (summer/winter).

These patterns are non-linear, so a simple straight-line regression won't capture them.

Polynomial regression lets you fit a smooth curve that adapts to rises and falls over the 366 days.

```
create_poly_plot <- function(degree, x, y) {
  # Fit polynomial model
  model <- lm(y ~ poly(x, degree, raw = TRUE))
  yfit <- predict(model, data.frame(x = x))

  rss <- sum(resid(model)^2)
  n <- 366
  aic <- n*log(rss/n)+2*degree

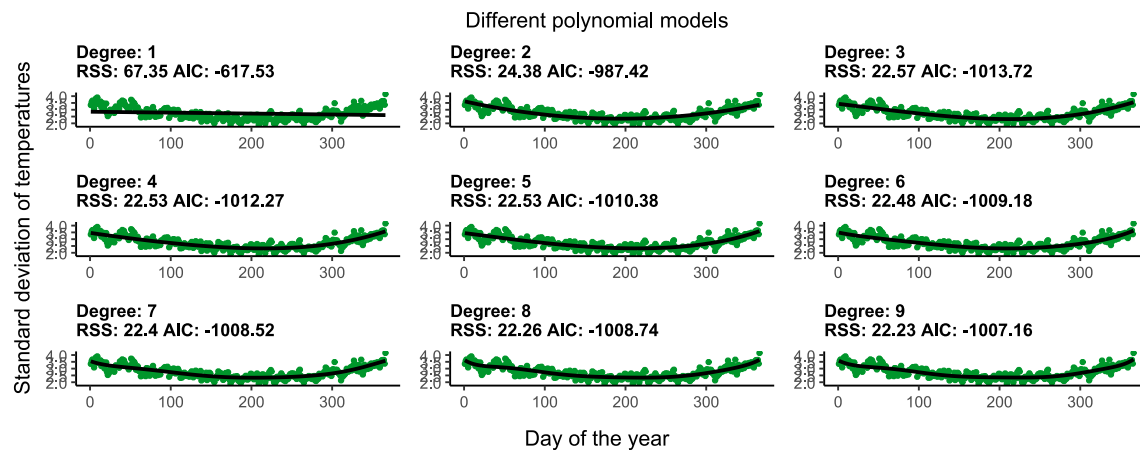
  plot <- ggplot() +
    geom_point(aes(x, y), color = '#00962B', size = 1) +
    geom_line(aes(x, yfit), color = 'black', linewidth = 1) +
    ggtitle(paste("Degree:", degree, "\nRSS:", round(rss, 2), "AIC:",
round(aic, 2))) +
    labs(y = NULL, x = NULL) +
    theme_classic() +
    theme(plot.title = element_text(size = 10, face = "bold"))

  return(list(
    plot = plot,
    degree = degree,
    rss = rss,
    aic = aic
  ))
}

degrees <- 1:9
results <- lapply(degrees, create_poly_plot, x = x, y = y)

# Extract plots
plots <- lapply(results, function(x) x$plot)

grid.arrange(grobs = plots, nrow = 3, ncol = 3, left = "Standard deviation of
temperatures", bottom = "Day of the year", top = "Different polynomial
models")
```



```
# Extract metrics into a data frame
poly_metrics_df <- data.frame(
  parameter = sapply(results, function(x) x$degree),
  rss = sapply(results, function(x) x$rss),
  aic = sapply(results, function(x) x$aic) %>%
    set_rownames(paste0("Polynomial", 1:9))
```

6.2 Fourier transforms

The Fourier transform is a mathematical technique that breaks down any signal (like your volatility over days) into a sum of sinusoids (sines and cosines) at different frequencies.

$$f(x) \approx a_0 + \sum_{k=1}^n (a_k \cos(2\pi kx/T) + b_k \sin(2\pi kx/T))$$

This is especially powerful for seasonal or periodic data like temperature volatility, because cycles are naturally captured by sines and cosines. This is especially powerful for seasonal or periodic data like temperature volatility, because cycles are naturally captured by sines and cosines.

for this application I'm not changing the Amplitude (Strength of each frequency component), or Phase (Timing offset of each component) opting to just increase the order

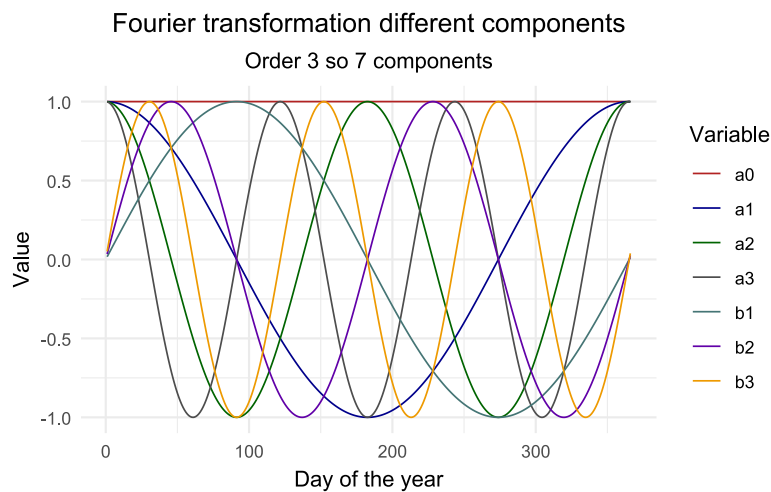
```
fourier_series <- function(x, n_terms, period = 365.25) {
  omega <- 2 * pi / period
  terms <- list(a0 = 1)

  for (i in 1:n_terms) {
    terms[[paste0("a", i)]] <- cos(i * omega * x)
    terms[[paste0("b", i)]] <- sin(i * omega * x)
  }
  return(terms)
}
```

```

fourier_series(1:366, 3) %>%
  data.frame() %>%
  quickplot(title = "Fourier transformation different components",
    subtitle = "Order 3 so 7 components",
    xlab = "Day of the year"
  )

```



```

create_fourier_plot <- function(n_terms, x, y) {
  # Fit Fourier series
  fourier_terms <- fourier_series(1:366, n_terms = n_terms)
  model_data <- data.frame(y = y, fourier_terms)

  fourier_fit <- lm(y ~ ., data = model_data)
  fourier <- predict(fourier_fit, newdata = model_data)

  rss <- sum(resid(fourier_fit)^2)
  n <- 366
  aic <- n*log(rss/n)+2*n_terms

  plot <- ggplot(data = data.frame(), aes(x = x)) +
    geom_point(aes(y = y), color = 'orangered3', size = 1) +
    geom_line(aes(y = fourier), color = 'black', linewidth = 1) +
    ggtitle(paste("Order:", n_terms, "\nRSS:", round(rss, 2), "AIC:",
      round(aic, 2))) +
    labs(y = NULL, x = NULL) +
    theme_classic()

  # Return both plot and metrics
  return(list(
    plot = plot,
    order = n_terms,

```

```

    rss = rss,
    aic = aic
  ))
}

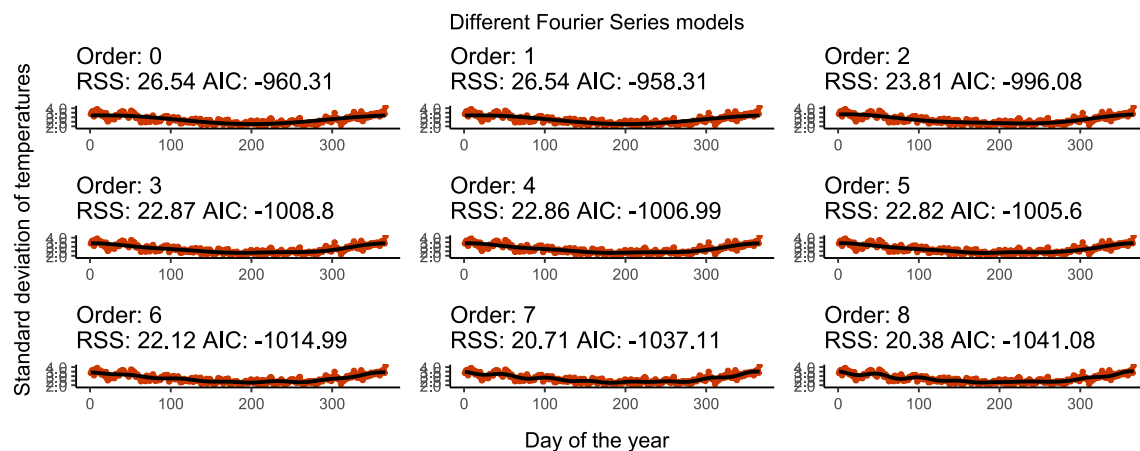
order <- c(0:8)

results <- lapply(order, create_fourier_plot, x = x, y = y)

plots <- lapply(results, function(x) x$plot)

grid.arrange(grobs = plots, nrow = 3, ncol = 3,
  left = "Standard deviation of temperatures",
  bottom = "Day of the year",
  top = "Different Fourier Series models")

```



```

# Extract metrics into a data frame
fourier_metrics_df <- data.frame(
  parameter = sapply(results, function(x) x$order),
  rss = sapply(results, function(x) x$rss),
  aic = sapply(results, function(x) x$aic)) %>%
  set_rownames(paste0("Fourier", 1:9))

```

6.3 B-splines

B-splines create smooth curves by joining together piecewise polynomials at specific points called “knots.” Think of it as drawing a smooth curve through control points, in this case Equally spaced knots, where each piece is a simple polynomial. It is a model with excellent numerical stability (very hard to create overfitting)

```

library(splines)
create_spline_plot <- function(knots, x, y) {

```

```

# Fit the spline model
spline_model <- lm(y ~ bs(x, knots = knots))
yfit <- predict(spline_model, data.frame(x = x))

rss <- sum((y - yfit)^2)
aic <- length(x) * log(rss / length(x)) + 2 * knots

plot <- ggplot() +
  geom_point(aes(x, y), color = 'cornflowerblue', size = 1.5) +
  geom_line(aes(x, yfit), color = 'black', linewidth = 1) +
  ggtitle(paste("Knots #", knots, "\nRSS:", round(rss, 2), "AIC:",
round(aic,2))) +
  labs(x = NULL, y = NULL) +
  theme_classic()

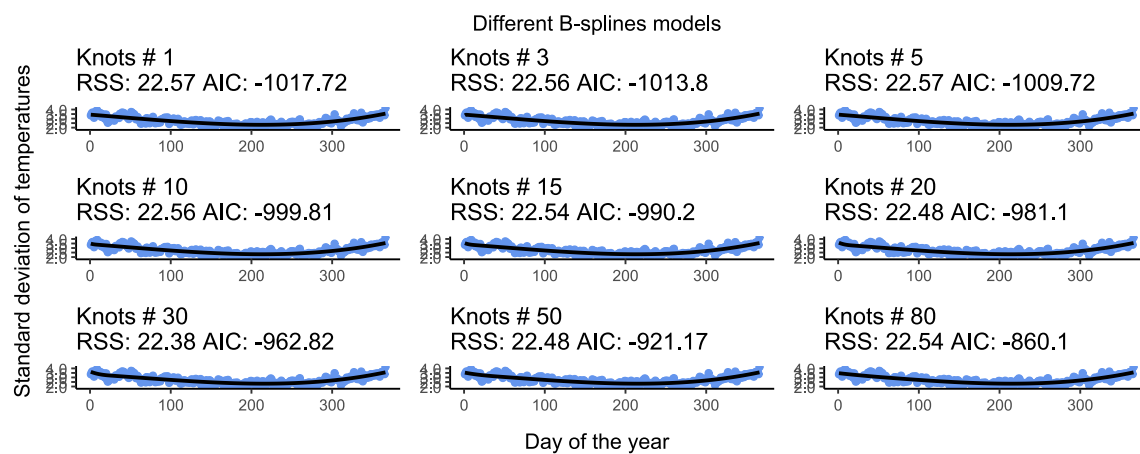
# Return both plot and metrics
return(list(
  plot = plot,
  knots = knots,
  rss = rss,
  aic = aic
))
}

knots <- c(1, 3, 5, 10, 15, 20, 30, 50, 80)
results <- lapply(knots, create_spline_plot, x = x, y = y)

plots <- lapply(results, function(x) x$plot)

grid.arrange(grobs = plots, nrow = 3, ncol = 3, left = "Standard deviation of
temperatures", bottom = "Day of the year", top = "Different B-splines models")

```



```
spline_metrics_df <- data.frame(
  parameter = sapply(results, function(x) x$knots),
  rss = sapply(results, function(x) x$rss),
  aic = sapply(results, function(x) x$aic)
) %>%
  set_rownames(paste0("B-Spline", 1:9))
```

6.4 Kernel smoothing

Kernel regression creates smooth predictions by taking weighted averages of nearby data points. Points closer to the prediction location get higher weights, creating a locally-adapted smooth curve, also known as the Nadaraya-Watson estimator.

$$\hat{y}(x) = \sum_i^n w_i(x) * y_i$$

where $w(x)$ is the weights function in this case I'm using a gaussian kernel $K(u) = \exp(-\frac{1}{2}u^2)$ the bandwidth (h) is the control term Small h: More wiggly, follows data closely (high variance, low bias) Large h: Smoother, averages over more points (low variance, high bias)

```
kernel_smooth <- function(x_eval, x_data, y_data, h_val=10) {
  weights <- exp(-0.5 * ((x_eval - x_data) / h_val)^2)
  weights <- weights / sum(weights)
  return(sum(weights * y_data))
}

create_kernel_plot <- function(x, y, bandwidth) {
  y_pred <- sapply(x, function(xi) kernel_smooth(xi, x, y, bandwidth))

  rss <- sum((y - y_pred)^2)
  n <- length(y)
  aic <- n * log(rss/n) + 2*bandwidth

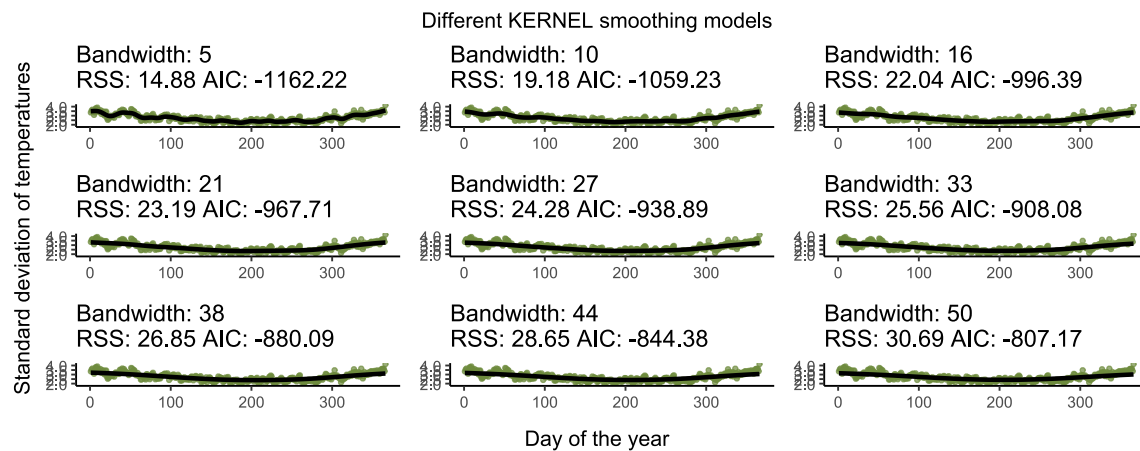
  data_plot <- data.frame(x = x, y = y, y_pred = y_pred)

  plot <- ggplot(data_plot, aes(x = x)) +
    geom_point(aes(y = y), color = 'darkolivegreen4', size = 1, alpha = 0.7) +
    geom_line(aes(y = y_pred), color = 'black', linewidth = 1.2) +
    ggtitle(paste("Bandwidth:", bandwidth, "\nRSS:", round(rss, 2), "AIC:",
    round(aic, 2))) +
    labs(y = NULL, x = NULL) +
    theme_classic()
  return(list(plot = plot, rss = rss, aic = aic, bandwidth = bandwidth))
}

width <- floor(seq(5, 50, length.out = 9))
results <- lapply(width, create_kernel_plot, x = x, y = y)
```

```
# Extract plots
plots <- lapply(results, function(x) x$plot)

grid.arrange(grobs = plots, nrow = 3, ncol = 3,
             left = "Standard deviation of temperatures",
             bottom = "Day of the year",
             top = "Different KERNEL smoothing models")
```



```
# Extract metrics into data frame
kernel_metrics_df <- data.frame(
  parameter = sapply(results, function(x) x$bandwidth),
  rss = sapply(results, function(x) x$rss),
  aic = sapply(results, function(x) x$aic)) %>%
  set_rownames(paste0("Kernel-smoothing", 1:9))
```

6.5 Loess modelling

LOESS fits a smooth curve by performing many small linear or quadratic regressions on local neighborhoods of data points. At each point, it gives more weight to nearby observations and less to distant ones. For each point x_0 , define a neighborhood containing the α fraction of closest points (span parameter)

- Small span: More wiggly, follows data closely
- Large span: Smoother, more global averaging

```
create_loess_plot <- function(span_val, x, y) {
  # Fit LOESS model
  loess_fit <- loess(y ~ x, span = span_val)
  loess_pred <- predict(loess_fit, newdata = data.frame(x = x))

  rss <- sum((y - loess_pred)^2)
```

```

# Effective degrees of freedom from loess
df <- loess_fit$enp # "equivalent number of parameters"
n <- 366
aic <- n * log(rss/n) + 2 * df

plot <- ggplot(data = data.frame(x = x, y = y, fit = loess_pred), aes(x =
x)) +
  geom_point(aes(y = y), color = 'violetred4', size = 1) +
  geom_line(aes(y = fit), color = 'black', linewidth = 1) +
  ggtitle(paste("Span:", span_val, "\nRSS:", round(rss, 2), "AIC:",
round(aic, 2))) +
  labs(y = NULL, x = NULL) +
  theme_classic()

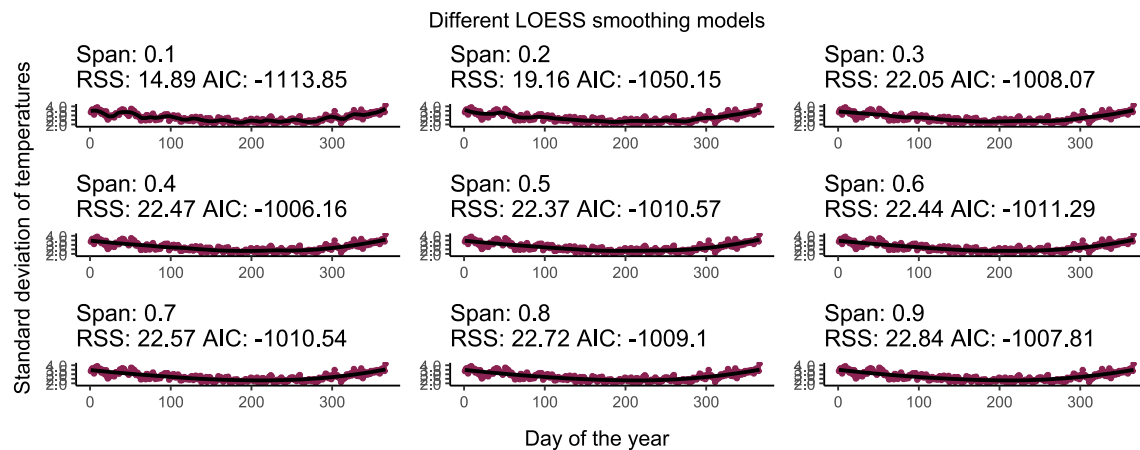
# Return both plot and metrics
return(list(
  plot = plot,
  span = span_val,
  rss = rss,
  aic = aic
))
}

spans <- seq(0.1, 0.9, by = 0.1)
results <- lapply(spans, create_loess_plot, x = x, y = y)

# Extract plots
plots <- lapply(results, function(x) x$plot)

grid.arrange(grobs = plots, nrow = 3, ncol = 3,
  left = "Standard deviation of temperatures",
  bottom = "Day of the year",
  top = "Different LOESS smoothing models")

```



```
# Extract metrics into data frame
loess_metrics_df <- data.frame(
  parameter = sapply(results, function(x) x$span),
  rss = sapply(results, function(x) x$rss),
  aic = sapply(results, function(x) x$aic)) %>%
  set_rownames(paste0("LOESS",1:9))
```

6.6 GAM

GAM models fit smooth, flexible curves to data by combining multiple smooth functions instead of assuming a specific parametric form. Think of it as “flexible regression” that can capture complex patterns without forcing them into straight lines or simple curves. A basis functions is needed to control the curvature, I’m using as the basis functions cyclic cubic regression splines `bs = "cc"`, a penalized cubic regression splines up to second derivative.

$$y = \beta_0 + f_1(x_1) + f_2(x_2) + \varepsilon$$

```
library(mgcv, warn.conflicts = F, quietly = T)
```

This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.

```
create_gam_plot <- function(x, y, k_value) {
  # Fit GAM with cyclic smooth for seasonal data
  gam_model <- gam(y ~ s(x, bs = "cc", k = k_value), weights = rep(1,
length(y)))
  y_pred <- predict(gam_model)

  rss <- sum((y - y_pred)^2)
  n <- 366
  edf <- sum(gam_model$edf) # effective degrees of freedom
  aic <- n * log(rss/n) + 2 * edf
```

```

data_plot <- data.frame(x = x, y = y, y_pred = y_pred)

plot <- ggplot(data_plot, aes(x = x)) +
  geom_point(aes(y = y), color = 'purple4', size = 1, alpha = 0.7) +
  geom_line(aes(y = y_pred), color = 'black', linewidth = 1.2) +
  ggtitle(paste("K:", k_value, "\nRSS:", round(rss, 2),
               "AIC:", round(aic, 2))) +
  theme_classic() +
  labs(y = NULL, x = NULL)

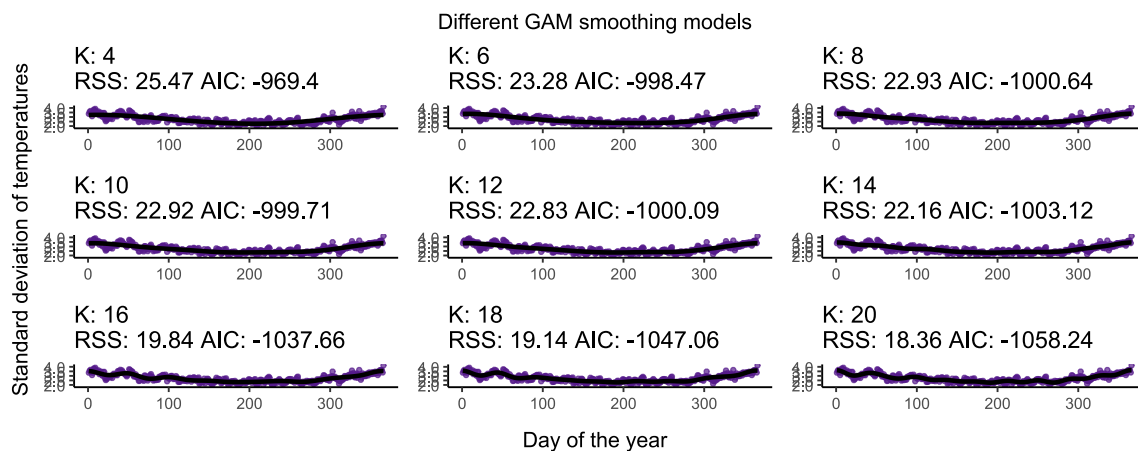
return(list(plot = plot, k_value = k_value, rss = rss, aic = aic))
}

k_values <- floor(seq(4, 20, length.out = 9))
results <- lapply(k_values, create_gam_plot, x = x, y = y)

# Extract plots
plots <- lapply(results, function(x) x$plot)

grid.arrange(grobs = plots, nrow = 3, ncol = 3,
             left = "Standard deviation of temperatures",
             bottom = "Day of the year",
             top = "Different GAM smoothing models")

```



```

# Extract metrics into data frame
gam_metrics_df <- data.frame(
  parameter = sapply(results, function(x) x$k_value),
  rss = sapply(results, function(x) x$rss),
  aic = sapply(results, function(x) x$aic) %>%
    set_rownames(paste0("GAM", 1:9))

```

6.7 Figuring out which is the best model

Now for the task of comparing the models as i

```
comparison_df <- rbind(
  poly_metrics_df,
  fourier_metrics_df,
  spline_metrics_df,
  kernel_metrics_df,
  loess_metrics_df,
  gam_metrics_df
) %>%
  set_colnames(c("Parameter", "RSS", "AIC"))

summary_comparison_df <- data.frame()

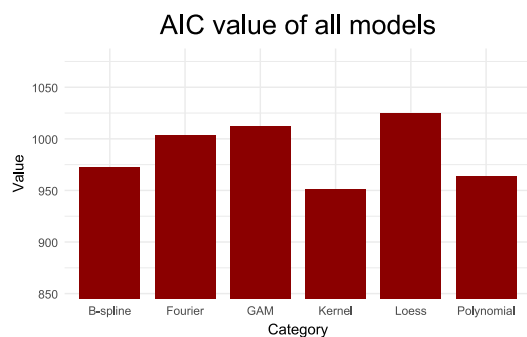
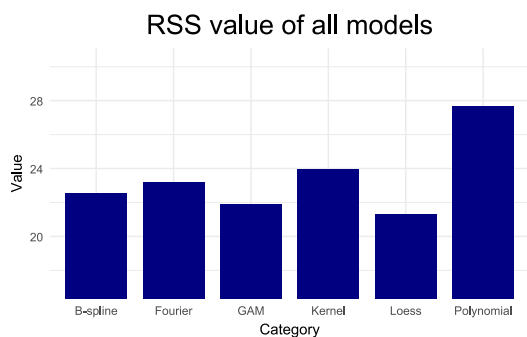
for (i in 1:6) {
  summary_comparison_df <- rbind(
    summary_comparison_df,
    (apply(comparison_df[seq((i - 1) * 9 + 1, 9 * i), ], 2, mean))
  )
}
summary_comparison_df[, 1] <- c(
  "Polynomial",
  "Fourier",
  "B-spline",
  "Kernel",
  "Loess",
  "GAM"
)
colnames(summary_comparison_df) <- c("Model", "RSS", "AIC")
summary_comparison_df$AIC <- abs(summary_comparison_df$AIC)

summary_comparison_df$RSS %>%
  set_names(summary_comparison_df$Model) %>%
  quickbarplot(title = "RSS value of all models", palette = "navyblue") +
  theme(plot.title = element_text(size = 20)) +
  coord_cartesian(
    ylim = c(
      min(summary_comparison_df$RSS) * 0.8,
      max(summary_comparison_df$RSS) * 1.1
    )
  )
summary_comparison_df$AIC %>%
  set_names(summary_comparison_df$Model) %>%
  quickbarplot(title = "AIC value of all models", palette = "darkred") +
  theme(plot.title = element_text(size = 20)) +
```

```

coord_cartesian(
  ylim = c(
    min(summary_comparison_df$AIC) * 0.9,
    max(summary_comparison_df$AIC) * 1.05
  )
)
head(comparison_df[order(comparison_df$AIC), ]) %>%
  gt(rownames_to_stub = T) %>%
  opt_stylize(5, color = "blue") %>%
  tab_header("Best model according to AIC")
head(comparison_df[order(comparison_df$RSS), ]) %>%
  gt(rownames_to_stub = T) %>%
  opt_stylize(5, color = "green") %>%
  tab_header("Best model according to RSS")
best_model <- comparison_df[which.min(comparison_df$AIC), ]
best_model_name <- gsub("[0-9]", "", (rownames(best_model)))
parameter <- switch(best_model_name,
  "Polynomial" = "degrees",
  "Fourier" = "order",
  "B-Spline" = "knots",
  "Kernel-smoothing" = "bandwidth",
  "LOESS" = "span",
  "GAM" = "k values",
  stop("Unknown model: ", best_model_name) # error
)

```



Best model according to AIC				Best model according to RSS			
	Parameter	RSS	AIC		Parameter	RSS	AIC
Kernel-smoothing	bandwidth1	14.87746	-1162.220	Kernel-smoothing	bandwidth1	14.87746	-1162.220
LOESS1	0.1	14.88682	-1113.848	LOESS1	0.1	14.88682	-1113.848
Kernel-smoothing	bandwidth2	19.18078	-1059.233	GAM9	20.0	18.36132	-1058.245
GAM9	20.0	18.36132	-1058.245	GAM8	18.0	19.14058	-1047.058
LOESS2	0.2	19.15993	-1050.153	LOESS2	0.2	19.15993	-1050.153
GAM8	18.0	19.14058	-1047.058	Kernel-smoothing	bandwidth2	19.18078	-1059.233

so the best model is Kernel-smoothing with bandwidth = 5. it has the lowest AIC at -1162.22 value and the best RSS at 14.88. so moving forward I will use that for the volatility modeling.

```
# 3. Print long-term volatility metrics
cat("Trend or long term volatility ~", round(mean(vol_df$std, na.rm = TRUE),
3), "\n")
```

```
Trend or long term volatility ~ 2.73
```

```
cat("Gamma is ~", round(sd(vol_df$std, na.rm = TRUE), 3), "\n")
```

```
Gamma is ~ 0.436
```

```
# 4. Fit AR(1) model for mean reversion rate
ar_model <- arima(vol_df$std, order = c(1, 0, 0), include.mean = FALSE)
coef <- ar_model$coef
residuals <- ar_model$residuals

cat("Rate of mean reversion of volatility process is:", round(coef["ar1"], 3),
"\n")
```

```
Rate of mean reversion of volatility process is: 0.998
```

```
summary(ar_model)
```

```
Call:
```

```

arima(x = vol_df$std, order = c(1, 0, 0), include.mean = FALSE)

Coefficients:
      ar1
      0.9977
s.e.    0.0026

sigma^2 estimated as 0.05128:  log likelihood = 21.58,  aic = -39.15

Training set error measures:
              ME      RMSE      MAE      MPE      MAPE      MASE
Training set 0.009025571 0.226445 0.1775325 -0.04251562 6.595296 1.001041
              ACF1
Training set -0.1478413

```

7 Montecarlo simulations

With the volatility component now modelled and a satisfactory result pulled from it it's time to move to the actual Montecarlo simulations for the temperature come the maturity date next year so starting from our model both the deterministic component as well as the volatility component need to be simulated and added to create the full simulated dataset from all our previous work using the Euler scheme approximation step in our modified Ornstein-Uhlenbeck model for temperature simulations

$$dT_t = \left(\frac{dT_{bar}(t)}{dt} + \kappa(T_{bar}(t) - T_t) \right) dt + \sigma(t) dW_t$$

```

a <- MOD_params[1]
b <- MOD_params[2]
theta <- atan2(MOD_params[3], MOD_params[4])
alpha <- sqrt(MOD_params[3]^2 + MOD_params[4]^2)
kappa <- as.double(kappa)

data.frame(a = a, b = b, theta = theta, alpha = alpha, kappa = kappa) %>%
  set_rownames("value")

```

```

      a      b      theta      alpha      kappa
value 9.532908 9.788622e-05 -2.217757 8.539706 0.226139

```

```

# Temperature Model Functions
T_model <- function(x, a, b, alpha, theta) {
  omega <- 2 * pi / 365.25
  a + b * x + alpha * sin(omega * x + theta)
}

```

```

dT_model <- function(x, a, b, alpha, theta) {
  omega <- 2 * pi / 365.25
  b + alpha * omega * cos(omega * x + theta)
}

# Apply Model with Given Parameters
Tbar_params <- list(a = a, b = b, alpha = alpha, theta = theta)
temp_t$ordinal <- as.numeric(temps$DAY) - as.numeric(temps$DAY[1]) # x values
first_ordinal_val <- as.numeric(temps$DAY[1])

temps$model_fit <- T_model(
  temp_t$ordinal,
  Tbar_params$a,
  Tbar_params$b,
  Tbar_params$alpha,
  Tbar_params$theta
)

```

At this point we care more about the distribution than any single point not if there is auto-correlation in the residuals

```

# Big switch function to get the correct model VALUES
fit_volatility <- switch(
  best_model_name,
  "Polynomial" = {
    yfit <- predict(lm(y ~ poly(x, parameter, raw = TRUE)), data.frame(x = x))
  },
  "Fourier" = {
    model_data <- data.frame(y = y, fourier_series(1:366, n_terms =
parameter))
    fourier_fit <- lm(y ~ ., data = model_data)
    yfit <- predict(fourier_fit, newdata = model_data)
  },
  "B-Spline" = {
    yfit <- predict(lm(y ~ bs(x, knots = parameter)), data.frame(x = x))
  },
  "Kernel-smoothing" = {
    yfit <- sapply(x, function(xi) kernel_smooth(xi, x, y, ))
  },
  "LOESS" = {
    loess_fit <-
    yfit <- predict(
      loess(y ~ x, span = parameter),
      newdata = data.frame(x = x)
    )
  },
  "GAM" = {

```

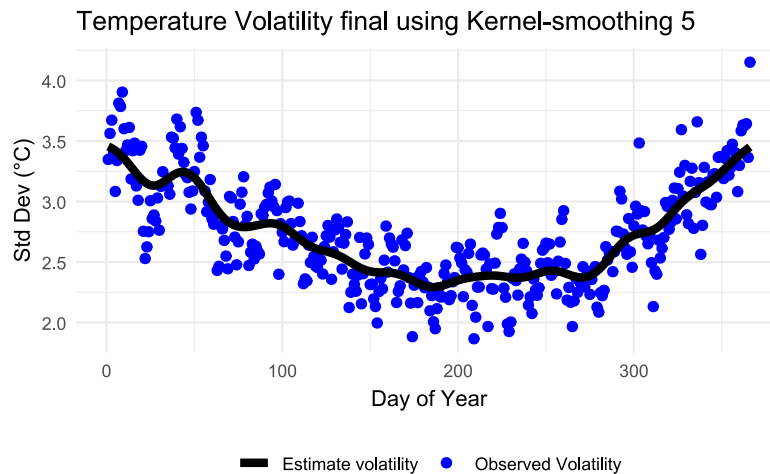
```

yfit <- predict(gam(
  y ~ s(x, bs = "cc", k = parameter),
  weights = rep(1, length(y))
))
},
# Default: error if unknown type
stop("Unknown model type: ", best_model_name)
)
vol_df$fvol <- yfit

# Plot Volatility
ggplot(vol_df, aes(x = day)) +
  geom_point(aes(y = std, color = "Observed Volatility"), size = 2) +
  geom_line(aes(y = fvol, color = "Estimate volatility"), size = 2) +
  scale_color_manual(
    NULL,
    values = c("Observed Volatility" = "blue", "Estimate volatility" =
"black")
  ) +
  labs(
    title = paste(
      "Temperature Volatility final using",
      best_model_name,
      best_model$Parameter
    ),
    y = "Std Dev (°C)",
    x = "Day of Year"
  ) +
  theme_minimal() +
  theme(legend.position = "bottom")

```

Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
i Please use `linewidth` instead.



Finally we can implement the functions for the simulation

```
# REAL Monte Carlo Simulation Functions
euler_step <- function(row, kappa, M) {
  T_i <- ifelse(is.na(row$Tbar_shift), row$Tbar, row$Tbar_shift)
  T_det <- T_i + row$dTbar
  T_mrev <- kappa * (row$Tbar - T_i)
  sigma <- row$vol * rnorm(M)
  T_det + T_mrev + sigma
}

monte_carlo_temp <- function(trading_dates, Tbar_params, vol_df,
                             first_ordinal, M = 1, kappa = 0.226139) {
  # Convert dates to numeric if needed
  trading_numeric <- as.numeric(trading_dates)
  kappa_val <- as.double(kappa)

  # Calculate Tbar and dTbar
  x_vals <- trading_numeric - first_ordinal_val
  Tbars <- T_model(
    x_vals,
    Tbar_params$a,
    Tbar_params$b,
    Tbar_params$alpha,
    Tbar_params$theta
  )

  dTbars <- dT_model(
    x_vals,
    Tbar_params$a,
    Tbar_params$b,
    Tbar_params$alpha,

```

```

    Tbar_params$theta
  )

  # Create simulation dataframe
  mc_temps <- data.frame(
    Date = trading_dates,
    Tbar = Tbars,
    dTbar = dTbars,
    day = yday(trading_dates),
    vol = data.frame(vol_df)$fvol[yday(trading_dates)] # Directly add
volatility
  )

  # Add lagged Tbar
  mc_temps$Tbar_shift <- dplyr::lag(mc_temps$Tbar)

  # Run simulations - modified apply call
  simulations <- sapply(1:nrow(mc_temps), function(i) {
    row <- mc_temps[i, ]
    euler_step(row, kappa_val, M)
  })

  # Transpose and format results
  simulations <- t(simulations)
  colnames(simulations) <- paste0("Sim", 1:M)

  list(
    mc_temps = mc_temps,
    mc_sims = cbind(Date = trading_dates, as.data.frame(simulations))
  )
}

```

I then decided to check performance in a couple of ways, first just a one year completed model to see how the performance fairs against both the stricter 5-95 percentile and also a fair but more generous $\pm 2\sigma$ from the simulated volatility.

```

# Run Simulation
one_year_trading_dates <- seq(as.Date("2024-12-31"), as.Date("2025-12-31"), by
= "day")
sim_results <- monte_carlo_temp(
  one_year_trading_dates,
  Tbar_params,
  vol_df,
  first_ordinal_val,
  M = 5
)

```

```

piv_sim <- pivot_longer(sim_results$mc_sims, -Date, names_to = "Simulation",
  values_to = "Temperature")

MIN_MAX <- data.frame(
  "MIN" = apply_convolution(apply(pivot_df[-1], 1, quantile, 0.05, na.rm
= TRUE), dnorm(-3:3)),
  "MAX" = apply_convolution(apply(pivot_df[-1], 1, quantile, 0.95, na.rm =
TRUE), dnorm(-3:3))
)

suppressWarnings(
  ggplot() +
    geom_ribbon(data = MIN_MAX, aes(x = one_year_trading_dates, ymin = MIN, ymax
= MAX), , alpha = 0.2) +
    geom_ribbon(data = MIN_MAX, aes(x = one_year_trading_dates, ymin =
MIN-2*vol_df$fvol, ymax = MAX+2*vol_df$fvol), alpha = 0, color = "red") +
    geom_point(data = piv_sim, aes(x = Date, y = Temperature, color =
Simulation), alpha = 0.7) +
    labs(title = "Monte Carlo Temperature Simulations", y = "Temperature (°C)", x
= "Day of the year") +
    guides(fill = "none") +
    theme(legend.position = "bottom"))

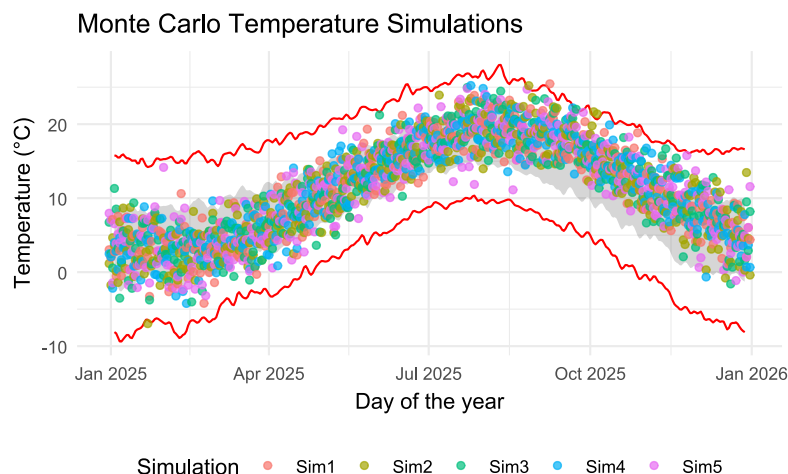
```

Don't know how to automatically pick scale for object of type <ts>. Defaulting to continuous.

Don't know how to automatically pick scale for object of type <ts>. Defaulting to continuous.

Don't know how to automatically pick scale for object of type <ts>. Defaulting to continuous.

Don't know how to automatically pick scale for object of type <ts>. Defaulting to continuous.



They are scattered across the historical envelope, which suggests:

- The simulations are plausible given past variability.
- There's no extreme bias (e.g., all simulations too hot or too cold).
- The model accounts for natural variability and uncertainty.
- The simulated values mostly stay within the historical bounds (plausible since we are taking the effects of climate change variations)
- A slight seasonal shift with spring and summer starting a bit later than historically

Finally we can check the distribution of the future datapoints and check it against the historical performance, we can see that the distribution takes on a different shape from the historical models there appears to be a reasonably good fit for your temperature forecast model:

- The general shape and spread of both distributions are well captured
- The seasonal separation is clearly maintained
- The peak positions are close to the historical data
- The overall temperature ranges match well

Areas where the fit could be improved:

Your simulated winter distribution appears slightly more peaked/narrow than the historical winter data, suggesting the model may be underestimating temperature variability in winter The simulated summer distribution looks quite close to historical, which is good The tails of the distributions seem reasonably well represented

This is what goes on behind the scenes split into different components

```
trading_dates_year <- seq(as.Date("2024-12-31"), as.Date("2025-12-31"), by =
"day")

x_vals <- as.numeric(trading_dates_year) - as.numeric(trading_dates_year)[1]
Tbar_vals <- T_model(x_vals, a, b, alpha, theta)
dTbar_vals <- dT_model(x_vals, a, b, alpha, theta)
sigma_vals <- data.frame(vol_df)$fvol[yday(trading_dates_year)]

data.frame(
  x_vals = x_vals,
  Tbar_vals = Tbar_vals,
  dTbar_vals = dTbar_vals,
  sigma_vals = sigma_vals
) %>%
  show_df()
```

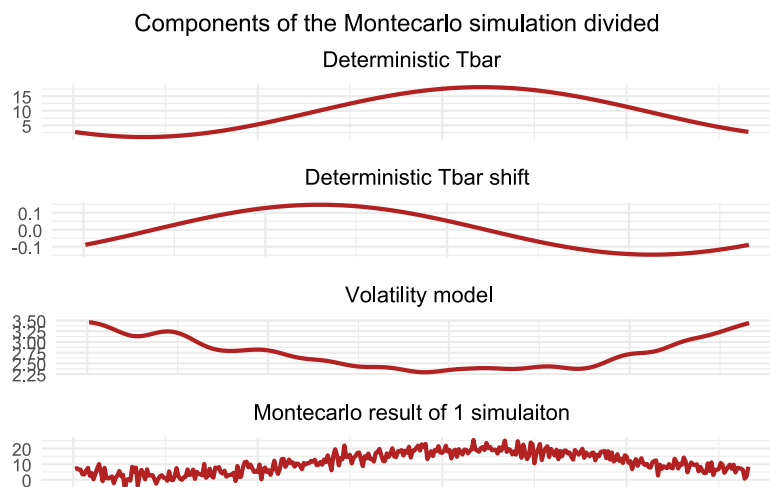
	DATE	x_vals	Tbar_vals	dTbar_vals	sigma_vals
1	1	0	2.718912	-0.08845040	3.450745
2	2	1	2.631474	-0.08642098	3.455030
3	3	2	2.546078	-0.08436595	3.448123
4	4	3	2.462750	-0.08228593	3.440245

5	5	4	2.381515	-0.08018153	3.431344
6	NA	NA	NA	NA	NA
7	362	361	3.148446	-0.09677601	3.392572
8	363	362	3.052625	-0.09486200	3.404770
9	364	363	2.958732	-0.09291989	3.416690
10	365	364	2.866794	-0.09095026	3.428326
11	366	365	2.776840	-0.08895368	3.439678

```

grid.arrange(top = "Components of the Montecarlo simulation divided", ncol =
1,
  quickplot(Tbar_vals, subtitle = "Deterministic Tbar", show_legend = F,
linewidth = 1, show_x = F, xlab = NULL, ylab = NULL),
  quickplot(dTbar_vals, subtitle = "Deterministic Tbar shift", show_legend =
F, linewidth = 1, show_x = F, xlab = NULL, ylab = NULL),
  quickplot(sigma_vals, subtitle = "Volatility model", show_legend = F,
linewidth = 1, show_x = F, xlab = NULL, ylab = NULL),
  quickplot(monte_carlo_temp(trading_dates_year, Tbar_params, vol_df,
first_ordinal_val, M = 2)$mc_sims[2], subtitle = "Montecarlo result of 1
simulaiton", show_legend = F, linewidth = 1, show_x = F, xlab = F, ylab =
NULL))

```



```

# Set number of simulations
no_sims <- 100000

# Define winter and summer dates
trading_dates_winter <- as.Date("2025-04-01")
trading_dates_summer <- as.Date("2025-10-01")

# Run simulations
sim_results_winter <- monte_carlo_temp(trading_dates_winter, Tbar_params,

```

```

vol_df, first_ordinal_val, M = no_sims)
sim_results_summer <- monte_carlo_temp(trading_dates_summer, Tbar_params,
vol_df, first_ordinal_val, M = no_sims)

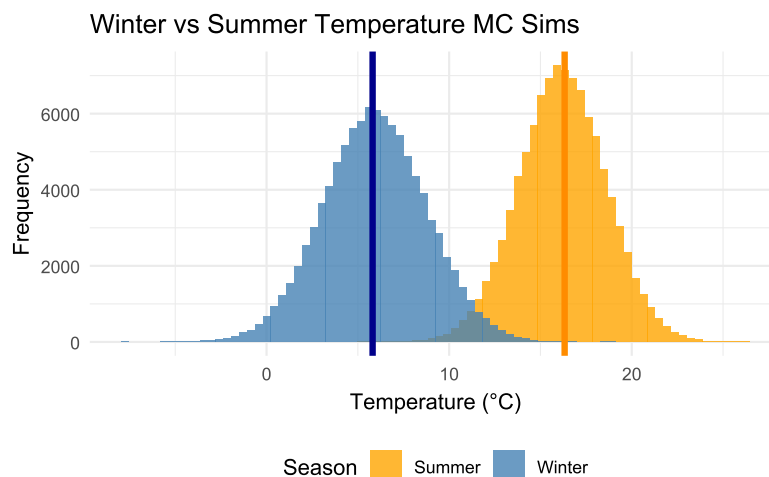
# Extract results
mc_sims_winter <- sim_results_winter$mc_sims %>% select(-Date)
mc_sims_summer <- sim_results_summer$mc_sims %>% select(-Date)

Tbar_summer <- sim_results_summer$mc_temps$Tbar[1]
Tbar_winter <- sim_results_winter$mc_temps$Tbar[1]

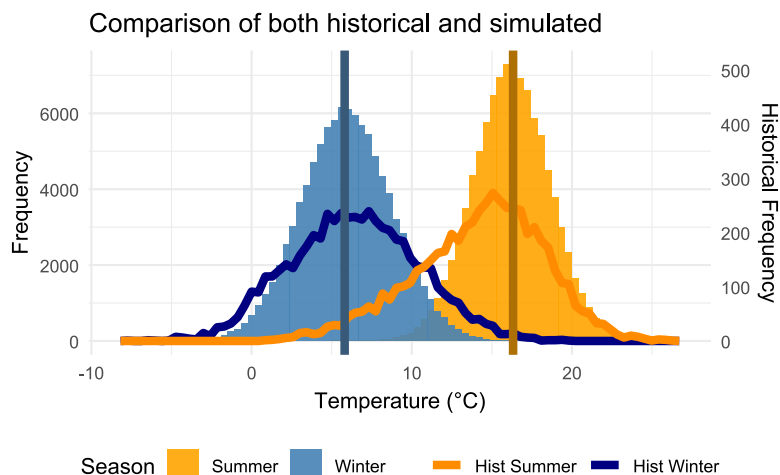
# Create combined data frame for plotting
plot_data <- bind_rows(
  data.frame(Temperature = unlist(mc_sims_summer), Season = "Summer"),
  data.frame(Temperature = unlist(mc_sims_winter), Season = "Winter")
)

# Create the plot
ggplot(plot_data, aes(x = Temperature, fill = Season)) +
  geom_histogram(position = "identity", alpha = 0.8, bins = 80) +
  geom_vline(aes(xintercept = Tbar_winter), color = "darkblue",
    linewidth = 1.5, linetype = "solid") +
  geom_vline(aes(xintercept = Tbar_summer), color = "darkorange",
    linewidth = 1.5, linetype = "solid") +
  scale_fill_manual(values = c(Winter="steelblue", Summer="orange")) +
  labs(title = "Winter vs Summer Temperature MC Sims",
    x = "Temperature (°C)",
    y = "Frequency") +
  theme_minimal() +
  theme(legend.position = "bottom")

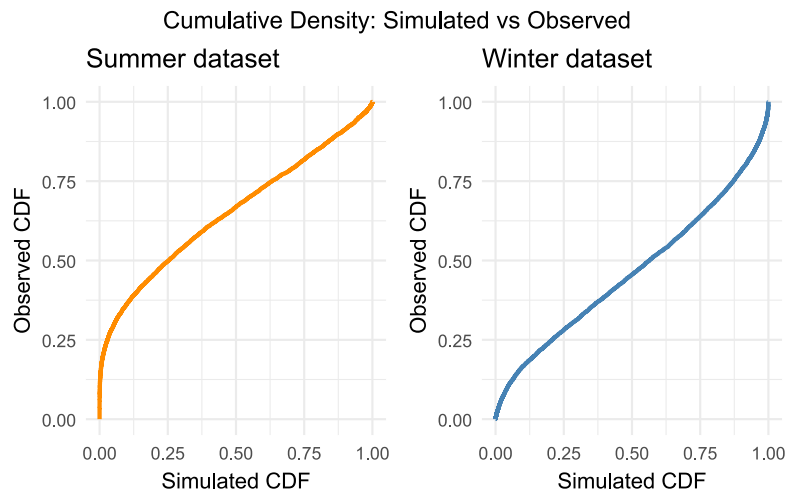
```



```
# Create the plot
ggplot(plot_data, aes(x = Temperature, fill = Season)) +
  geom_histogram(position = "identity", alpha = 0.9, bins = 80) +
  geom_freqpoly(data=winter_dataset, aes(x = T_AVG, y = after_stat(count*10),
color = "Hist Winter"), bins = 80, linewidth = 2)+
  geom_freqpoly(data=summer_dataset, aes(x = T_AVG, y = after_stat(count*10),
color = "Hist Summer"), bins = 80, linewidth = 2)+
  geom_vline(aes(xintercept = Tbar_winter), color = darken("steelblue", 0.3),
linewidth = 2, linetype = "solid") +
  geom_vline(aes(xintercept = Tbar_summer), color = darken("orange", 0.3),
linewidth = 2, linetype = "solid") +
  scale_fill_manual(values = c(Winter = "steelblue", Summer = "orange")) +
  scale_color_manual(name = NULL, values = c("Hist Winter" = "navy", "Hist
Summer" = "darkorange")) +
  labs(title = "Comparison of both historical and simulated", x = "Temperature
(°C)", y = "Simulated Frequency") +
  theme(legend.position = "bottom") +
  scale_y_continuous(name = "Frequency", sec.axis = sec_axis(~ . * 0.07, name
= "Historical Frequency"))
```



```
grid.arrange( top = "Cumulative Density: Simulated vs Observed", nrow = 1,
suppressWarnings(compare_cdf(unlist(mc_sims_summer), summer_dataset$T_AVG,
"Summer dataset", "darkorange")),
suppressWarnings(compare_cdf(unlist(mc_sims_winter), winter_dataset$T_AVG,
"Winter dataset", "steelblue")))
```



8 Risk neutral pricing

London's weather is rather chilly, so, we need to actually structure a contract that makes sense

So we need to compute the probability that the option does not pay out (i.e., temperature T_n stays above 18°C).

The smooth probability decay from 99.995% at 6°C to just 0.142% at 28°C indicates your model properly captures the tail behavior of summer heat, with meaningful probabilities of extreme warm weather that would trigger cooling degree day payouts.

```
temps_checked <- 5:28
prob_no_payout_df <- data.frame(matrix(numeric(length(temps_checked)*3), ncol
= 3)) %>%
  set_colnames(c("temps", "summer", "winter"))
prob_no_payout_df[, 1] <- temps_checked

prob_sim_summer <- as.numeric(sim_results_summer$mc_sims[-1])
counter <- 1
for (n in temps_checked) {
  payoffs <- ifelse(prob_sim_summer >= n, 0, n - prob_sim_summer)
  prob_no_payout <- mean(payoffs == 0) * 100
  prob_no_payout_df[counter, 2] <- prob_no_payout
  cat(paste0("Probability P(max(", n, "-Tn, 0) = 0): ", prob_no_payout,
"%", "\n"))
  counter <- counter+1
}
```

```
Probability P(max(5-Tn, 0) = 0): 100%
Probability P(max(6-Tn, 0) = 0): 99.998%
Probability P(max(7-Tn, 0) = 0): 99.994%
```

```

Probability P(max(8-Tn, 0) = 0): 99.965%
Probability P(max(9-Tn, 0) = 0): 99.865%
Probability P(max(10-Tn, 0) = 0): 99.533%
Probability P(max(11-Tn, 0) = 0): 98.504%
Probability P(max(12-Tn, 0) = 0): 96.053%
Probability P(max(13-Tn, 0) = 0): 91.087%
Probability P(max(14-Tn, 0) = 0): 82.287%
Probability P(max(15-Tn, 0) = 0): 69.396%
Probability P(max(16-Tn, 0) = 0): 53.442%
Probability P(max(17-Tn, 0) = 0): 36.989%
Probability P(max(18-Tn, 0) = 0): 22.79%
Probability P(max(19-Tn, 0) = 0): 12.042%
Probability P(max(20-Tn, 0) = 0): 5.498%
Probability P(max(21-Tn, 0) = 0): 2.238%
Probability P(max(22-Tn, 0) = 0): 0.785%
Probability P(max(23-Tn, 0) = 0): 0.219%
Probability P(max(24-Tn, 0) = 0): 0.048%
Probability P(max(25-Tn, 0) = 0): 0.009%
Probability P(max(26-Tn, 0) = 0): 0.001%
Probability P(max(27-Tn, 0) = 0): 0%
Probability P(max(28-Tn, 0) = 0): 0%

```

The winter distribution presents a stark contrast, demonstrating the model's ability to differentiate seasonal risk profiles effectively. Winter temperatures show virtually no probability of staying above most thresholds until the temperature reach the 20-25°C range, reflecting the reality that winter weather derivatives would almost certainly pay out for heating degree days. The extremely low probabilities of no payout (0% for most reasonable thresholds) until temperatures exceed 20°C confirms the model captures the fundamental asymmetry between seasonal risks. This distribution pattern suggests the temperature simulation successfully models the heavy lower tail of winter temperatures while maintaining realistic upper bounds, creating the foundation for properly pricing temperature derivatives that protect against seasonal weather extremes.

```

prob_sim_winter <- as.numeric(sim_results_winter$mc_sims[-1])

counter <- 1
for (n in temps_checked) {
  payoffs <- ifelse(prob_sim_winter <= n, 0, n - prob_sim_winter)
  prob_no_payout <- mean(payoffs == 0) * 100
  cat(paste0("Probability P(max(",n,"-Tn, 0) = 0): ", prob_no_payout,
"%", "\n"))
  prob_no_payout_df[counter,3] <- prob_no_payout
  counter <- counter+1
}

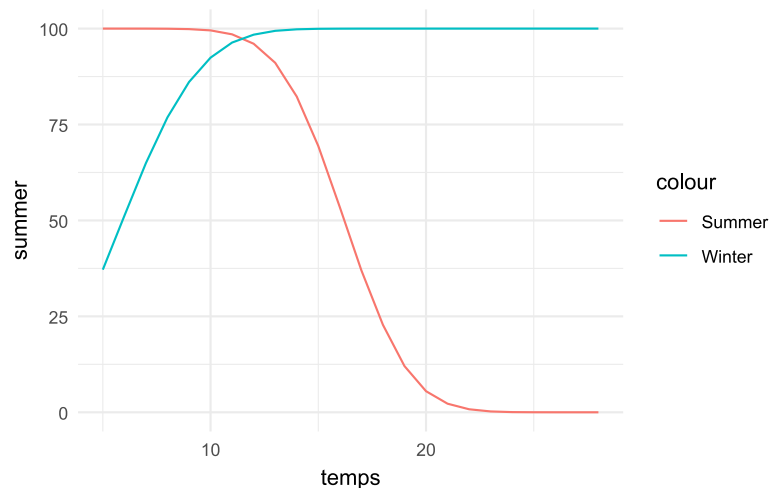
```

Probability $P(\max(5-T_n, 0) = 0)$: 37.173%
 Probability $P(\max(6-T_n, 0) = 0)$: 51.199%
 Probability $P(\max(7-T_n, 0) = 0)$: 64.943%
 Probability $P(\max(8-T_n, 0) = 0)$: 76.868%
 Probability $P(\max(9-T_n, 0) = 0)$: 86.08%
 Probability $P(\max(10-T_n, 0) = 0)$: 92.43%
 Probability $P(\max(11-T_n, 0) = 0)$: 96.365%
 Probability $P(\max(12-T_n, 0) = 0)$: 98.416%
 Probability $P(\max(13-T_n, 0) = 0)$: 99.417%
 Probability $P(\max(14-T_n, 0) = 0)$: 99.811%
 Probability $P(\max(15-T_n, 0) = 0)$: 99.947%
 Probability $P(\max(16-T_n, 0) = 0)$: 99.985%
 Probability $P(\max(17-T_n, 0) = 0)$: 99.997%
 Probability $P(\max(18-T_n, 0) = 0)$: 99.997%
 Probability $P(\max(19-T_n, 0) = 0)$: 99.999%
 Probability $P(\max(20-T_n, 0) = 0)$: 100%
 Probability $P(\max(21-T_n, 0) = 0)$: 100%
 Probability $P(\max(22-T_n, 0) = 0)$: 100%
 Probability $P(\max(23-T_n, 0) = 0)$: 100%
 Probability $P(\max(24-T_n, 0) = 0)$: 100%
 Probability $P(\max(25-T_n, 0) = 0)$: 100%
 Probability $P(\max(26-T_n, 0) = 0)$: 100%
 Probability $P(\max(27-T_n, 0) = 0)$: 100%
 Probability $P(\max(28-T_n, 0) = 0)$: 100%

```

ggplot(prob_no_payout_df, aes(x = temps)) +
  geom_line(aes(y = summer, color = "Summer")) +
  geom_line(aes(y = winter, color = "Winter"))

```



8.1 Market model

The market for weather derivatives is a typical example of an incomplete market, because the underlying variable, the temperature, is not tradable. Therefore we have to consider the market price of risk λ , in order to obtain unique prices for such contracts.

Since there is not yet a real market from which we can obtain prices, we assume for simplicity that the market price of risk is constant. Furthermore, we assume that we are given a risk free asset with constant interest rate r and a contract that for each degree Celsius pays one unit of currency. Thus, under a martingale measure $\mathbb{Q}$, characterized by the market price of risk λ , our price process also denoted by T_t satisfies the following dynamics:

So under risk neutral probability it becomes

$$dT_t^{\mathbb{Q}} = \left(\frac{d\bar{T}_t}{dt} + \kappa(\bar{T}_t - T_t) - \lambda\sigma_t \right) dt + \sigma_t dW_t^{\mathbb{Q}}$$

Compared to the physical probability measure

$$dT_t^{\mathbb{P}} = \left(\frac{d\bar{T}_t}{dt} + \kappa(\bar{T}_t - T_t) \right) dt + \sigma_t dW_t^{\mathbb{P}}$$

Only losing the $-\lambda\sigma_t$ term

For simplicity's sake i'm just doing Montecarlo valuation as some other researchers did not find much difference with analytical valuation of the Black and Scholes closed form stochastic calculus method

```
temperature_option <- function(trading_dates, no_sims, Tbar_params, vol_df,
r = 0.05, alpha = 2500, K = 300, T_maturity = 1, first_ordinal =
first_ordinal_val, option_type = "call", ref_temperature = 18) {
  T_maturity_mod <- (max(as.numeric(trading_dates)) -
min(as.numeric(trading_dates)))/365.25
  if (T_maturity==1) {
    T_maturity <- T_maturity_mod
  }
  # Run Monte Carlo temperature simulation (assumes your R version exists)
  mc_res <- monte_carlo_temp(trading_dates, Tbar_params, vol_df,
first_ordinal, no_sims)
  mc_sims <- mc_res$mc_sims # matrix/dataframe of simulated temps

  M <- ncol(mc_sims) # number of simulations
  N <- nrow(mc_sims) # number of time steps (not directly needed)

  mc_arr <- as.matrix(mc_sims[, -1])

  # Cooling degree days (CDD) or similar metric
  DD <- colSums(pmax(ref_temperature - mc_arr, 0))
```

```

# Option payoff
if (option_type == "call") {
  Payoff <- alpha * pmax(DD - K, 0)
} else {
  Payoff <- alpha * pmax(K - DD, 0)
}

# Discounted expected payoff
C0 <- exp(-r * T_maturity) * mean(Payoff)

# Standard error of estimator
sigma <- sqrt(sum((exp(-r * T_maturity) * Payoff - C0)^2) / (M - 1))
SE <- sigma / sqrt(M)

return(list(price = C0, se = SE))
}

trading_dates <- seq(as.Date("2024-12-01"), as.Date("2025-03-01"), by = "day")

temperature_option(trading_dates, 10000, Tbar_params, vol_df, r = 0.05, alpha
= 2500, K = 300, T_maturity = 1, first_ordinal, option_type = "put",
ref_temperature = 18) %>% bprint()

```

```

price = 0
se = 0

```

```

temperature_option(trading_dates, 10000, Tbar_params, vol_df, r = 0.05, alpha
= 2500, K = 300, T_maturity = 1, first_ordinal, option_type = "call",
ref_temperature = 18) %>% bprint()

```

```

price = 2410503
se = 763.3597

```

```

cat("maturity =", (max(as.numeric(trading_dates)) -
min(as.numeric(trading_dates)))/365.25, "years")

```

```

maturity = 0.2464066 years

```

```

strikes <- seq(450, 650, length.out = 20)
ref_t <- 10
nsim <- 10000
alpha <- 5000

```

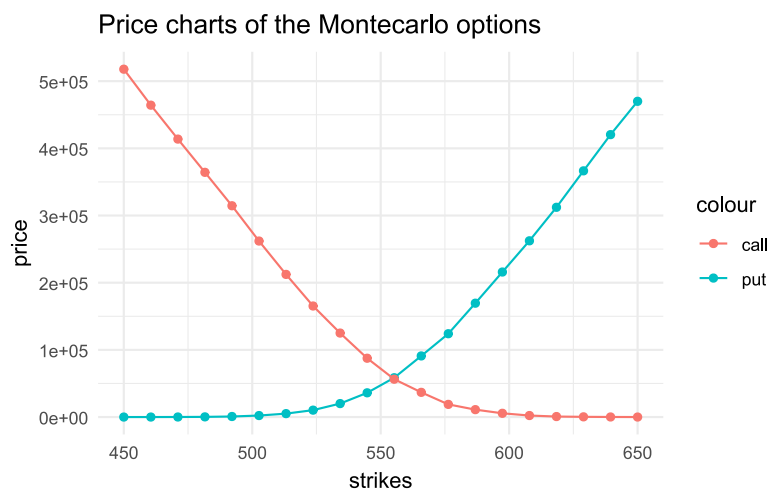
```

prices_options <- data.frame(matrix(numeric(length(strikes)*4),ncol = 4)) %>%
  set_colnames(c("put", "call", "SE_put", "SE_call"))
counter <- 1
for (i in strikes) {
  PUT <- temperature_option(trading_dates, nsim, Tbar_params, vol_df,
    ref_temperature = ref_t, K = i, option_type = "put", alpha = alpha)
  CALL <- temperature_option(trading_dates, nsim, Tbar_params, vol_df,
    ref_temperature = ref_t, K = i, option_type = "call", alpha = alpha)

  prices_options[counter,1] <- PUT$price
  prices_options[counter,2] <- CALL$price
  prices_options[counter,3] <- PUT$se
  prices_options[counter,4] <- CALL$se
  counter <- counter+1
}

ggplot(cbind(prices_options,strikes), aes(x = strikes))+
  geom_line(aes(y = put, color = "put")) +
  geom_point(aes(y = put, color = "put")) +
  geom_line(aes(y = call, color = "call")) +
  geom_point(aes(y = call, color = "call")) +
  labs(title = "Price charts of the Montecarlo options", y = "price")

```



```

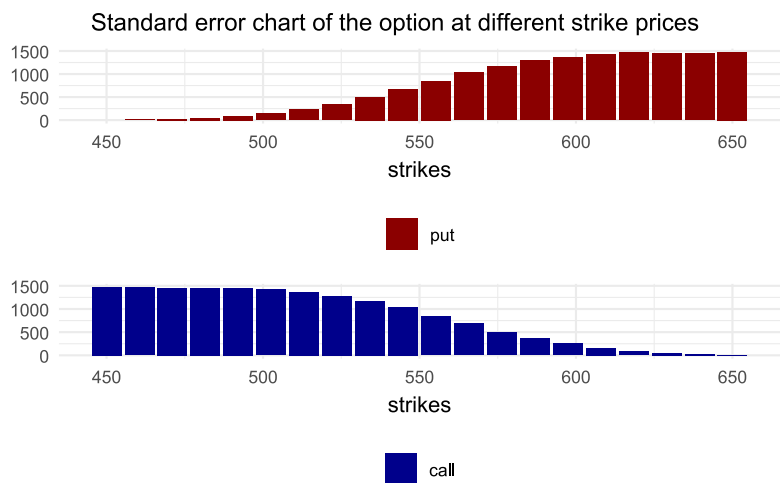
grid.arrange(top = "Standard error chart of the option at different strike
prices",
  ggplot(cbind(prices_options,strikes), aes(x = strikes))+
    geom_col(aes(y = SE_put, fill = "put")) +
    scale_fill_manual(NULL, values = "darkred") +
    ylab(NULL) +
    theme(legend.position = "bottom"),

```

```

ggplot(cbind(prices_options,strikes), aes(x = strikes))+
  geom_col(aes(y = SE_call, fill = "call"))+
  ylab(NULL) +
  scale_fill_manual(NULL, values = "darkblue")+
  theme(legend.position = "bottom")
)

```



```

# Add this debug code:
mc_res <- monte_carlo_temp(trading_dates, Tbar_params, vol_df, first_ordinal,
100)
mc_sims <- mc_res$mc_sims # matrix/dataframe of simulated temps
M <- ncol(mc_sims) # number of simulations
N <- nrow(mc_sims) # number of time steps (not directly needed)
mc_arr <- as.matrix(mc_sims[, -1])

dd_sample <- colSums(pmax(ref_t - mc_arr, 0))
cat("DD stats: mean =", mean(dd_sample), ", range =", range(dd_sample), "\n")

```

```
DD stats: mean = 550.7064 , range = 485.2438 615.5707
```

```
cat("Strikes in range?", any(strikes > min(dd_sample) & strikes <
max(dd_sample)), "\n")
```

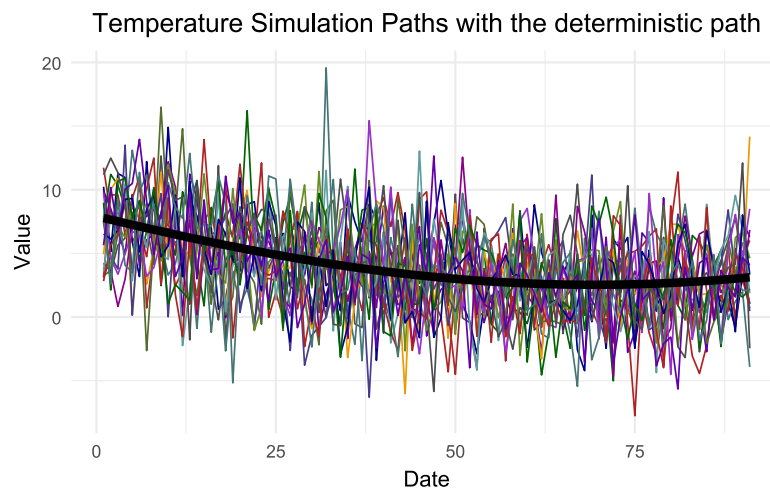
```
Strikes in range? TRUE
```

```

quickplot(mc_res$mc_sims[, -1][1:20], show_legend = F,
  title = "Temperature Simulation Paths with the deterministic path") +

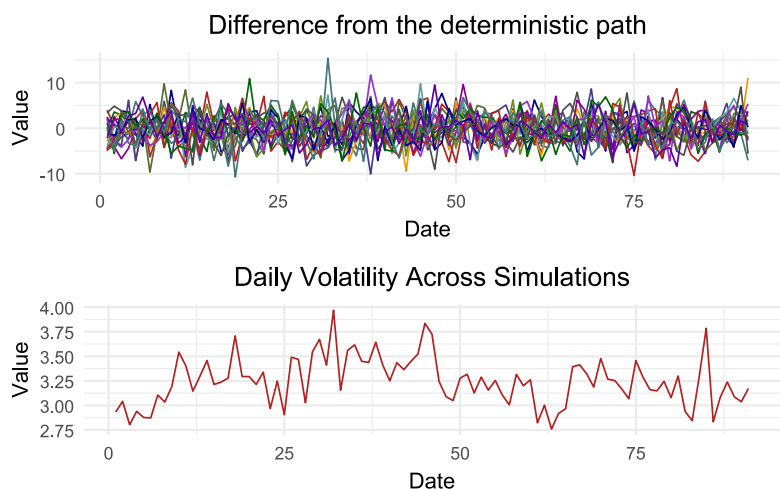
```

```
geom_line(
  data = data.frame(x = 1:length(trading_dates), y = mc_res$mc_temps$Tbar),
  aes(x = x, y = y),
  color = "black",
  linewidth = 2
)
```



```
temp_diffs <- apply(mc_arr, 2, function(x) x - mc_res$mc_temps$Tbar)[, 1:20]
daily_vol <- apply(mc_arr, 1, sd)

grid.arrange(
  quickplot(temp_diffs, title = "Difference from the deterministic path",
    show_legend = F),
  quickplot(daily_vol, title = "Daily Volatility Across Simulations",
    show_legend = F)
)
```



```
cat("Mean deviation from seasonal average:", mean(abs(temp_diffs)), "\n")
```

```
Mean deviation from seasonal average: 2.603168
```

```
cat("Volatility range:", range(daily_vol), "\n")
```

```
Volatility range: 2.759051 3.968004
```

8.2 Greek Numerical Approximations

Since we aren't dealing with plain vanilla options the only option to calculate geeks is to do them via Numerical Approximations using:

$$\text{Greek} \approx \frac{V(x + \varepsilon) - V(x - \varepsilon)}{2\varepsilon}$$

Where:

x is the input variable ε is a small shift

This is a finite difference approach: generalizable and useful when analytical Greeks are unavailable

This method works for any option and it still assumes that the function is smooth and approximable locally, like the ones coming from the closed form solution of the Black and Scholes model but it breaks down with large shocks.

All this has been done with an $\alpha = 1$ to not have a magnifying effect on the value due to the price per tick being generally very expensive

- Greeks give powerful local sensitivities

- Good for quick P&L estimates, hedging, and risk management

Temperature Sensitivity Delta Shock the baseline temperature parameter 'a' akin to a change in price

```
shock <- 0.1
params_up <- Tbar_params
params_down <- Tbar_params
params_up$a <- Tbar_params$a + shock
params_down$a <- Tbar_params$a - shock

price_up <- temperature_option(trading_dates, 1000, params_up, vol_df,
option_type = "call", alpha = 1)$price
price_down <- temperature_option(trading_dates, 1000, params_down, vol_df,
option_type = "call", alpha = 1)$price

delta <- (price_up - price_down) / (2 * shock)

data.frame(price_up, price_down, delta) %>%
  gt() %>% cols_label(price_up = "Price up", price_down = "Price down", delta
= "Delta") %>%
  opt_stylize(5) %>% tab_header(paste("Delta with shock =", shock))
```

Delta with shock = 0.1

Price up	Price down	Delta
955.4433	972.6391	-85.97874

Volatility Sensitivity Vega Achieved by changing the fitted volatility function

```
shock <- 0.1
vol_up <- vol_df
vol_down <- vol_df
vol_up$fvol <- vol_df$fvol + shock
vol_down$fvol <- vol_df$fvol - shock

price_up <- temperature_option(trading_dates, 1000, params_up, vol_up,
option_type = "call", alpha = 1)$price
price_down <- temperature_option(trading_dates, 1000, params_down, vol_down,
option_type = "call", alpha = 1)$price

vega <- (price_up - price_down) / (2 * shock)

data.frame(price_up, price_down, vega) %>%
  gt() %>% cols_label(price_up = "Price up", price_down = "Price down", vega =
"Vega") %>%
  opt_stylize(5) %>% tab_header(paste("Vega with shock =", shock))
```

Vega with shock = 0.1

Price up	Price down	Vega
954.2237	973.4831	-96.29715

Seasonality Sensitivity (Alpha Sensitivity) Sensitivity to seasonal amplitude changes Where the negative shock means that the data is more seasonal

```
shock <- 0.1
params_up <- Tbar_params
params_down <- Tbar_params
params_up$alpha <- Tbar_params$alpha + shock
params_down$alpha <- Tbar_params$alpha - shock

price_up <- temperature_option(trading_dates, 1000, params_up, vol_df,
option_type = "call", alpha = 1)$price
price_down <- temperature_option(trading_dates, 1000, params_down, vol_df,
option_type = "call", alpha = 1)$price

alpha_g <- (price_up - price_down) / (2 * shock)

data.frame(price_up, price_down, alpha_g) %>%
  gt() %>% cols_label(price_up = "Price up", price_down = "Price down",
alpha_g = "Alpha greek") %>%
  opt_stylize(5) %>% tab_header(paste("Seasonality greek with shock =", shock))
```

Seasonality greek with shock = 0.1

Price up	Price down	Alpha greek
970.2211	956.1018	70.59645

Reference Temperature Greek Now we are more in the realm of contract modifications which are not possible but still greek as we are just changing a parameter

```
shock <- 3
price_up <- temperature_option(trading_dates, 1000, Tbar_params, vol_df,
option_type = "call", alpha = 1, ref_temperature = 18+shock)$price
price_down <- temperature_option(trading_dates, 1000, Tbar_params, vol_df,
option_type = "call", alpha = 1, ref_temperature = 18-shock)$price

temp_g <- (price_up - price_down) / (2 * shock)

data.frame(price_up, price_down, temp_g) %>%
  gt() %>% cols_label(price_up = "Price up", price_down = "Price down", temp_g
= "Reference T Greek") %>%
```

```
opt_stylize(5) %>% tab_header(paste("Reference Temperature Greek with shock =", shock))
```

Reference Temperature Greek with shock = 3

Price up	Price down	Reference T Greek
1234.448	695.0401	89.90134

Time passing Sensitivity (Θ) This is probably the most out there since unlike financial options where time decay is purely about discounting, temperature derivatives have path-dependent payoffs. When you shorten the period, you're not just reducing time to expiration - you're fundamentally changing the underlying (total degree days accumulated).

```
days_elapsed <- 3

price_t0 <- temperature_option(trading_dates, 1000, Tbar_params, vol_df,
option_type = "call", alpha = 1)$price

# Option after some days have passed (shorter remaining period)
remaining_dates <- trading_dates[(days_elapsed):length(trading_dates)]
price_t1 <- temperature_option(remaining_dates, 1000, Tbar_params, vol_df,
option_type = "call", alpha = 1)$price

theta <- (price_t1 - price_t0) / days_elapsed

data.frame(price_up, price_down, temp_g) %>%
  gt() %>% cols_label(price_up = "Price T0", price_down = "Price T1", temp_g =
"Reference T Greek") %>%
  opt_stylize(5) %>% tab_header(paste("Theta with shock =", days_elapsed))
```

Theta with shock = 3

Price T0	Price T1	Reference T Greek
1234.448	695.0401	89.90134

9 Sidequests

9.1 Sidequest: predicting temperatures in the future

Right now on 11-5-2025 i want to do an experiment and actually see if at least in some way i can predict the weather using this model that i've built right now i want to predict the temperature one month from now

```
model_formula <- function(t, a, b, a1, b1) {
  omega <- 2 * pi / 365.25
```

```

    a + b * t + a1 * cos(omega * t) + b1 * sin(omega * t)
  }

forecast_N <- 30
new_t <- max(temps$NUM_DAY) + forecast_N # forecast_N days after last
observation
predicted_T <- model_formula(new_t, MOD_params["a"], MOD_params["b"],
MOD_params["alpha"], MOD_params["theta"])
cat("predicted temperature for", as.character(max(temps$DAY)+forecast_N), "=",
predicted_T)

```

predicted temperature for 2025-10-09 = 15.22176

```

arma_model <- Arima(temps$RESID, order = c(2, 0, 0), include.mean = FALSE)
future_residuals <- forecast(arma_model, h = forecast_N) # forecast_N steps
ahead

first_date <- min(temps$DAY)
future_dates <- seq(max(temps$DAY), by = "day", length.out = forecast_N)
future_t <- as.numeric(difftime(future_dates, first_date, units = "days"))

# Deterministic part
deterministic_part <- model_formula(future_t, MOD_params["a"],
MOD_params["b"], MOD_params["alpha"], MOD_params["theta"])

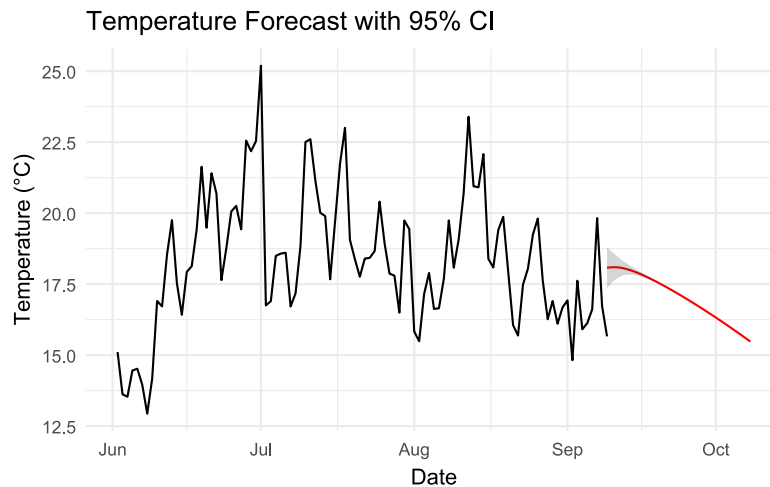
# Stochastic part (residuals)
stochastic_part <- future_residuals$mean

# Final prediction
future_T <- deterministic_part + stochastic_part

forecast_df <- data.frame(
  Date = future_dates,
  Temperature = future_T,
  Lower = future_T - 1.96 * future_residuals$mean, # 95% CI
  Upper = future_T + 1.96 * future_residuals$mean
)

ggplot() +
  geom_line(data = tail(temps, 100), aes(x = DAY, y = T_AVG)) +
  geom_line(data = forecast_df, aes(x = Date, y = Temperature), color = "red")
+
  geom_ribbon(data = forecast_df, aes(x = Date, ymin = Lower, ymax = Upper),
alpha = 0.2) +
  labs(title = "Temperature Forecast with 95% CI", y = "Temperature (°C)", x =
"Date")

```



The residuals represent short-term deviations from the deterministic trend/seasonality, becoming irrelevant after 27 days, which means that in the long run you will get a temperature entirely determined by the deterministic trend, over time, these deviations are expected to fade back to zero (the mean).

Forecast implications:

for forecasting purposes it means a couple of things: that in the short-term my Residuals adjust for recent “shocks” (e.g., a heatwave/cold snap), but in the long-term the forecast relies entirely on the deterministic component (trend + seasonality).

This model doesn’t really account for Volatility clustering, like how temperature tends to have time-varying volatility (e.g., summer vs. winter variability). which is the part of the OU that we still need to model

9.2 Milan comparison

i just moved from milan and was wondering about the distribution of climate here versus the one from milan

```
MILAN_RAW <- read.csv("Milan_DATA.csv", skip = 10)

MIL_DATASET <- MILAN_RAW %>%
  mutate(
    T_MAX = remove_outliers(T2M_MAX, fill = "NA") %>% na.approx(),
    T_MIN = remove_outliers(T2M_MIN, fill = "NA") %>% na.approx(),
    DAY   = as.Date(DOY - 1, origin = paste0(YEAR, "-01-01")),
    Month = month(DAY),
    T_AVG = (T_MAX + T_MIN) / 2
  ) %>%
  select(DAY, YEAR, Month, DOY, T_MAX, T_MIN, T_AVG)

# Pivot just once for Milan (average only)
```

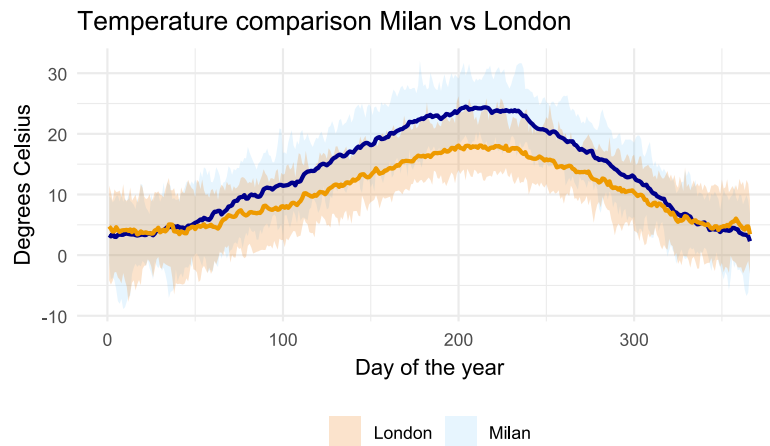
```

MIL_pivot_df <- MIL_DATASET %>%
  select(DOY, YEAR, T_AVG) %>%
  pivot_wider(names_from = YEAR, values_from = T_AVG)

# Build comparison dataset (Milan + London summary stats)
FD <- data.frame(
  DOY      = MIL_pivot_df$DOY,
  MIL_MIN  = apply(MIL_pivot_df[-1], 1, min, na.rm = TRUE),
  MIL_MAX  = apply(MIL_pivot_df[-1], 1, max, na.rm = TRUE),
  MIL_MEAN = apply(MIL_pivot_df[-1], 1, mean, na.rm = TRUE),
  LON_MIN  = apply(pivot_df[-1], 1, min, na.rm = TRUE),
  LON_MAX  = apply(pivot_df[-1], 1, max, na.rm = TRUE),
  LON_MEAN = apply(pivot_df[-1], 1, mean, na.rm = TRUE)
)

# Plot comparison
ggplot(FD, aes(x = DOY)) +
  geom_ribbon(aes(ymin = MIL_MIN, ymax = MIL_MAX, fill = "Milan"), alpha =
0.2) +
  geom_ribbon(aes(ymin = LON_MIN, ymax = LON_MAX, fill = "London"), alpha =
0.2) +
  geom_line(aes(y = MIL_MEAN, color = "Milan Avg"), linewidth = 1) +
  geom_line(aes(y = LON_MEAN, color = "London Avg"), linewidth = 1) +
  scale_color_manual(name = NULL, values = c("Milan Avg" = "darkblue", "London
Avg" = "orange2")) +
  scale_fill_manual(name = NULL, values = c(Milan = "lightskyblue", London =
"darkorange2")) +
  labs(
    title = "Temperature comparison Milan vs London",
    y = "Degrees Celsius",
    x = "Day of the year",
    caption = "NASA POWER - Data Access Viewer (DAV) 1982–2025"
  ) +
  theme(legend.position = "bottom") +
  guides(color = "none")

```



NASA POWER - Data Access Viewer (DAV) 1982–2025

```
minn <- min(FD$LON_MIN, FD$MIL_MIN)
maxx <- max(FD$LON_MAX, FD$MIL_MAX)

grid.arrange(
  top = "Boxplot for all months across the years",
  nrow = 1,
  DATASET %>%
    select(Month, T_AVG) %>%
    group_by(Month) %>%
    ggplot(aes(x = Month, y = T_AVG, group = Month)) +
    geom_boxplot(fill = "lightskyblue", outliers = FALSE, staplewidth = 0.8) +
    scale_x_continuous(breaks = seq(1, 12, by = 1), labels = month.abb) +
    labs(title = "London", x = NULL, y = NULL) +
    ylim(minn, maxx)
  ,

  MIL_DATASET %>%
    select(Month, T_AVG) %>%
    group_by(Month) %>%
    ggplot(aes(x = Month, y = T_AVG, group = Month)) +
    geom_boxplot(fill = "darkorange2", outliers = FALSE, staplewidth = 0.8) +
    scale_x_continuous(breaks = seq(1, 12, by = 1), labels = month.abb) +
    labs(title = "Milan", x = NULL, y = NULL) +
    ylim(minn, maxx)
)
```

Boxplot for all months across the years

